



Universidad Politécnica
de Madrid

**Escuela Técnica Superior de
Ingenieros Informáticos**



Máster Universitario en Artificial Intelligence

Trabajo Fin de Máster

**SmolVLA-MD: Integration of Short-Term
Memory and Depth Perception in VLA
Models for Robotic Manipulation**

Autor: Juan Esteban Rincón Marín
Tutor: Javier de Lope Asiaín

Madrid, July 2026

Este Trabajo Fin de Máster se ha depositado en la ETSI Informáticos de la Universidad Politécnica de Madrid para su defensa.

Trabajo Fin de Máster
Máster Universitario en Artificial Intelligence

*Título: SmolVLA-MD: Integration of Short-Term Memory and Depth Perception
in VLA Models for Robotic Manipulation*

July 2026

Autor: Juan Esteban Rincón Marín
Tutor: Javier de Lope Asiaín
Department of Artificial Intelligence
ETSI Informáticos
Universidad Politécnica de Madrid

To Santi
when you grow up I'll explain all this to you

Acknowledgments

“Once you’ve got a task to do, it’s better to do it than live with the fear of it.”

—Joe Abercrombie, *The Blade Itself*

“Only priests and fools are fearless and I’ve never been on the best of terms with God.”

—Patrick Rothfuss, *The Name of the Wind*

I want to thank from the bottom of my heart all the people who support me, especially when I dive headfirst into impossible challenges. To those who know and understand that I always embark on adventures with no guarantees, but trust in my determination. Thank you to those who have lent a hand to move this project forward and, above all, to those who have filled this journey with laughter and good times. Because, in the end, the journey is always more important than the destination.

Summary

Vision-Language-Action (VLA) models allow generating robotic actions directly from visual observations and natural language instructions. However, most current policies rely exclusively on RGB images from the current time step, which limits their capability in tasks involving temporal occlusions or three-dimensional geometry requirements. This Master’s Thesis investigates the integration of short-term visual memory and depth information in SmolVLA, a lightweight VLA model designed for robotic environments with limited computational resources. The proposal is implemented on a SO-101 robotic arm through imitation learning from teleoperated demonstrations. Four experimental variants are evaluated and compared: SmolVLA-Vanilla (baseline), SmolVLA-M (causal temporal attention), SmolVLA-D (geometric features injected into action expert layers selected after an ablation study), and SmolVLA-MD (joint fusion). Physical testing in scenarios with self-occlusions and out-of-distribution distractors reveals that the isolated incorporation of depth (SmolVLA-D) increases the mean normalized score from 0.55 to 0.86 in controlled environments, reducing grasping errors. However, this variant degrades substantially in the presence of visual distractors, while memory acts as a temporal stabilizer without resolving total loss of visibility. Furthermore, the direct combination (SmolVLA-MD) does not produce a positive synergy and interferes with the action expert’s dynamics, causing freezing and erratic trajectories. It is concluded that sensory expansion in compact VLA models requires adaptive multimodal fusion mechanisms and a more diverse training data corpus.

Abstract

Vision-Language-Action (VLA) models enable robotic action generation from visual inputs and text instructions, yet reliance on single-frame RGB observations limits performance under occlusions and 3D geometric demands. This Master’s Thesis evaluates integrating short-term visual memory and depth perception into SmolVLA, a lightweight VLA model. Deployed on an SO-101 robotic arm, four variants were tested: SmolVLA-Vanilla (baseline), SmolVLA-M (causal temporal attention), SmolVLA-D (point cloud features injected into ablation-selected action expert layers), and SmolVLA-MD (joint fusion). Real-world benchmarks show that depth alone (SmolVLA-D) increases mean normalized grasping scores from 0.55 to 0.86 in controlled setups, though it degrades under visual distractors. Temporal memory mainly serves as a trajectory stabilizer, while joint fusion (SmolVLA-MD) disrupts action expert dynamics, inducing freezes and erratic motions. Expanding sensory modalities in compact VLAs requires adaptive fusion mechanisms rather than direct concatenation.

Keywords: Robotics, Vision-Language-Action, SmolVLA, Visual Memory, Depth Perception, Imitation Learning.

Table of Contents

Acknowledgments	iii
Abstract	vii
List of Figures	xi
List of Tables	xiv
List of Abbreviations	xvii
1. Introduction	1
1.1. Main Objective	2
1.2. Specific Objectives	2
1.3. Working Hypotheses	2
1.4. Methodology and Work Plan	3
1.5. Document Structure	3
2. State of the Art	5
2.1. Precursors of the Current Paradigm	5
2.2. Birth and Consolidation of the VLA Paradigm	9
2.3. Generalization to New Environments and Platforms	13
2.4. Open Models and Efficiency	17
2.4.1. SmolVLA: Affordable Robotics	20
2.5. Tactile Perception, Memory, and Complex Tasks	22
2.6. Three-Dimensional Geometric Representation	27
2.7. Synthesis and Work Positioning	31
3. Reference Architecture and System Fundamentals	33
3.1. Imitation Learning	33
3.2. Flow Matching	34
3.3. SmolVLA Architecture	35
3.4. Three-Dimensional Representation and Feature Injection	37
3.5. Short-Term Visual Memory	37
4. Methodology	39
4.1. Experimental Environment	39
4.2. Data Collection and Preparation	41
4.3. SmolVLA Variants	42
4.3.1. SmolVLA-Vanilla	43

4.3.2. SmolVLA-M	43
4.3.3. SmolVLA-D	45
4.3.4. SmolVLA-MD	45
4.3.5. Trainings	46
4.4. Experimental Strategy	47
4.4.1. Layer Ablation Test	47
4.4.2. Final Evaluation	48
5. Results and Discussion	51
5.1. Layer Ablation Test Results	51
5.2. Final Evaluation Results	53
5.3. Discussion of the Final Evaluation	55
6. Conclusions and Future Work	57
6.1. Conclusions	57
6.2. Future Work	58
Bibliography	61

List of Figures

2.1. Qualitative examples of voxelized observations and action predictions in PerAct. The figure shows different manipulation scenes, along with the action predicted by the model and the reference expert action, both indicated on the three-dimensional representation of the environment. The examples on the left correspond to simulated tasks and those on the right to real tasks, adapted from [1].	6
2.2. Imitation learning process used in ALOHA and ACT. Demonstrations are collected through a bimanual teleoperation system and used to train a policy that generates action sequences from visual and proprioceptive observations [2].	7
2.3. Operation of action chunking and temporal ensembling in ACT. The policy predicts blocks of k future actions and combines the overlapping predictions corresponding to a single time step to obtain the finally executed action [2].	8
2.4. Conceptual diagram of RT-2. Continuous robotic actions are discretized and represented by tokens, which are generated autoregressively and subsequently converted into executable control commands [3].	10
2.5. Conceptual representation of Octo. Instructions, visual and proprioceptive observations, and representations used to generate actions are integrated through a common Transformer architecture, adaptable to different robotic configurations [4].	11
2.6. Conceptual diagram of Π_0 . The vision-language backbone processes images and instruction, while an action expert conditioned by its representations generates blocks of continuous actions [5].	12
2.7. Dual architecture of GR00T N1. The vision-language module, named System 2, interprets observations and the instruction, while System 1 generates motor actions via a Diffusion Transformer conditioned by the preceding representations [7].	14
2.8. Data pyramid used in GR00T N1. Web data and human videos provide the largest scale base; synthetic and simulated data occupy the intermediate level; and physical robot trajectories constitute the most highly specific embodied source [7].	15
2.9. Training and inference process of $\Pi_{0.5}$. Pre-training integrates heterogeneous sources using discretized actions and multimodal tasks; subsequently, a continuous action expert based on flow matching generates action blocks for robotic control [8].	16

2.10	Heterogeneous sources used in $\Pi_{0.5}$. The model combines robotic trajectories, subtask prediction, verbal instructions, object detection, and multimodal examples from the web to favor transfer to unobserved domestic environments [8].	17
2.11	OpenVLA architecture. Visual representations extracted via DINOv2 and SigLIP are projected into the Llama 2 space, which generates discretized tokens later converted into continuous control commands [6].	18
2.12	TinyVLA architecture and adaptation process. A compact vision-language model is combined with a diffusion policy that generates continuous action sequences, avoiding autoregressive prediction of control tokens [9].	19
2.13	General scheme of SmolVLA. The model combines a compact VLM with an action expert trained via flow matching and uses public datasets collected on affordable robotic platforms [10].	20
2.14	SmolVLA’s asynchronous inference pipeline. The generation of new action blocks overlaps with the execution of available blocks, reducing waiting periods between perception, prediction, and control [10].	21
2.15	Overview of VTLA. The model processes visual and tactile observations along with a linguistic instruction to generate actions in peg-in-hole insertion tasks [11].	23
2.16	VTLA training pipeline. In the first phase, the model is fine-tuned via next-token prediction. In the second, DPO introduces a preference-based signal to better approximate the continuous nature of robotic actions [11].	24
2.17	Overview of MEM. The system combines a short-term visual memory, based on video sequences, with a long-term text memory that summarizes semantic events relevant to the task [12].	26
2.18	Functional architecture of MEM. The high-level policy updates text memory and selects subtasks, while the low-level policy uses recent observations, visual memory, and semantic context to generate continuous actions [12].	26
2.19	3D Diffusion Policy architecture. Depth info transforms into a point cloud, gets processed by a lightweight encoder, and conditions a diffusion policy responsible for generating continuous action sequences [13].	28
2.20	General PointVLA architecture. The model preserves the pre-trained VLA’s vision-language processing and incorporates a point cloud representation via a 3D module connected to certain action expert blocks [14]. . . .	29
2.21	Skip-block analysis used in PointVLA. Action expert layer importance is evaluated via omission during inference. Less critical blocks are chosen to receive 3D info, reducing interference on pre-trained representations [14].	30
3.1.	Conceptual evolution of the flow matching process. Own elaboration based on [15].	34
3.2.	Simplified architecture of SmolVLA. The vision-language model processes RGB images, the instruction, and proprioception to generate a multimodal context that constantly conditions the action expert in action generation. Own elaboration based on [10].	36
4.1.	SO-101 arms for teleoperation: follower with gripper (left) and leader (right). Images from https://huggingface.co/docs/lerobot/so101 .	40

4.2. System cameras: fixed top RGB (left), monocular wrist RGB (center), and SVPRO stereo with its calibration and side-by-side capture (right). . . .	40
4.3. Workspace discretization: 3×3 grid (positions A1-C3) for object origin and fixed destination area.	41
4.4. Processing pipeline for the depth dataset: stereo view rectification, disparity map calculation, and point cloud reprojection.	42
4.5. Relationship and hierarchy of the SmolVLA model family developed from the base checkpoint.	42
4.6. General overview of the SmolVLA-Vanilla architecture.	44
4.7. SmolVLA-M architecture: integration of causal temporal attention in layers 3, 7, and 11 of the visual encoder.	44
4.8. SmolVLA-D architecture.	45
4.9. SmolVLA-MD architecture.	46
4.10 Evolution of training loss and learning rate during the training of the four final configurations. The vertical lines indicate the completion or interruption point of each run.	47
4.11 Evolution of training loss and learning rate during fine-tuning of the base model for the ablation study.	49
5.1. Mean normalized score and complete success rate obtained when individually disabling each action expert layer. Error bars represent the standard deviation of the score.	52
5.2. Comparison between the expected performance based on individual ablations and the observed performance when disabling several contiguous layers.	52
5.3. Distribution of failure modes observed in the base model, individual ablations, and combinations of contiguous layers.	53
5.4. Comparison of the mean normalized score by model and experimental condition (without photo and with photo).	53
5.5. Normalized score of each physical run per model. The vertical line separates episodes without a photograph from episodes with a photograph. The horizontal line represents the mean of that set of runs.	54
5.6. Behavioral metrics of the final evaluation: complete success, timeout, emergency intervention, recollection of the occluded object, finishing on the occluded object, and photograph rejection. Derived from execution dynamics and failure modes, these rates allow identifying exactly how each model behaves and why it fails.	55

List of Tables

2.1. Synthesis of the main works reviewed in the state of the art.	32
4.1. Summary of training configuration and execution for the final models (NVIDIA H100 80GB).	46
4.2. Configuration of the training used as the baseline for the ablation. . . .	48
4.3. Evaluation rubric breakdown according to scenario condition.	50

List of Abbreviations

Abbreviation	Meaning
---------------------	----------------

BC	Behavioral Cloning.
CNN	Convolutional Neural Network.
DOF	Degrees of Freedom.
DPO	Direct Preference Optimization.
ID	In-Distribution.
LLM	Large Language Model.
LoRA	Low-Rank Adaptation.
MEM	Multi-Scale Embodied Memory.
OOD	Out-of-Distribution.
RGB-D	Red, Green, Blue and Depth.
SO-101	Six-degree-of-freedom robotic platform used in this work.
VGTE	Vision-Guided Temporally Enhanced Tokens.
ViT	Vision Transformer.
VLA	Vision-Language-Action.
VLM	Vision-Language Model.

Capítulo 1

Introduction

Over the last few decades, industrial robotics has reached an impressive level of technological maturity, being able to operate with the highest level of precision and speed in structured environments with pre-programmed tasks. Automated assembly lines are a clear example of extraordinarily optimized systems, where robot arms execute millimetric trajectories predictably. However, the current challenge in robotics lies in creating control systems capable of handling the uncertainty of a less structured environment. The real obstacle right now does not lie in improving the physical capabilities of the robot, or equipping it with countless rigid rules to react to its environment. The obstacle lies in endowing it with the capabilities to interpret the environment, understand the tasks to be carried out, and adapt to the variability of the real world.

Advances to address this challenge rely on recent developments in deep learning, among which the Transformer architecture stands out for its ability to process and relate large-scale data sequences. Initially conceived for natural language processing, this architecture was extended to the visual domain through Vision Transformers (ViT), which represent images as sequences of tokens. Following this same approach, the architecture was transferred to robotics to train action generators conditioned by environment observations and the assigned task. In this way, Vision-Language-Action (VLA) models emerged, which allow the development of more flexible control policies capable of adapting to unstructured environments and a wider diversity of tasks.

However, robotics poses challenges that the Transformer architecture had not yet faced, such as the three-dimensional understanding of the scene or its temporal coherence. Real-world object manipulation requires more than just interpreting the environment and understanding the goal; it requires mastering the three-dimensional space through which the robot moves, estimating the volume of objects, and keeping a record of previous actions when the task demands a sequence of steps. Being a relatively recent approach, the question of which sensory improvements can help bridge this gap remains open.

In this context, the central question of this work arises: can an improvement in depth perception and short-term memory optimize the dexterity of a VLA model in object manipulation tasks? To answer this question, different variants developed from the base model are analyzed in a real physical environment. In this way, it is possible to evaluate the impact of each modification and attribute its improvements in isolation.

1.1. Main Objective

The main objective of this work is to design, implement, and evaluate a modular extension of SmolVLA that incorporates short-term visual memory and depth information. Through this extension, the aim is to study the effect of both sources of information on the behavior of a robotic manipulation policy, especially in situations of partial observability, self-occlusion, and spatial ambiguity.

The proposal, named SmolVLAMD, builds upon a lightweight and pre-trained Vision-Language-Action model, avoiding the development of a foundational policy from scratch. The work therefore focuses on analyzing how new perceptual and temporal capabilities can be integrated without replacing the visual, linguistic, and motor knowledge acquired by the base model.

1.2. Specific Objectives

To achieve the main objective, the following specific objectives are proposed:

- Analyze the SmolVLA architecture through an ablation study on its action expert to determine the optimal integration regions for the new components.
- Configure the experimental setup with the SO-101 robot to collect a custom dataset through teleoperated demonstrations.
- Design and implement the short-term visual memory mechanism and the depth data processing and injection module.
- Develop the four experimental variants of the model necessary to isolate and compare the impact of each sensory improvement.
- Establish a common evaluation protocol that considers both ID and OOD test scenarios.
- Compare the behavior of the different configurations with the base model to evaluate whether depth information and memory actually improve the robot's dexterity.

1.3. Working Hypotheses

To guide the experimental evaluation of this work, the following hypotheses are proposed:

- **H1 (Temporal memory):** The incorporation of short-term visual memory will allow retaining information from past observations, improving the policy's performance in situations of self-occlusion, temporary loss of visibility, and partial observability.
- **H2 (Spatial perception):** The integration of depth information will provide an explicit geometric representation of the scene, increasing the model's robustness in tasks sensitive to distance, volume, and the spatial location of objects.

- **H3 (Capability synergy):** The simultaneous combination of visual memory and depth information will offer superior performance to both the base model and its individual variants, by providing complementary temporal and geometric information.

1.4. Methodology and Work Plan

First, a state-of-the-art review of VLA models is conducted to understand where they come from, what techniques they use to generate actions, and what sensory improvements are currently being applied. This is complemented with the theoretical foundations necessary to understand how to integrate temporal and depth information, as well as how to obtain the training data.

Once the foundations of the problem have been reviewed and the limitations in current models identified, the development of this work’s main proposal is addressed. First, the experimental platform is configured, from which data collection via teleoperation is performed, and the test protocol in the physical environment is designed.

Following the data collection, the design and training of the improvements upon the base model proceed. To this end, a prior ablation study is included in order to find the optimal points where to integrate the new components. Finally, the four variants are evaluated on the experimental platform to analyze, in isolation, the impact of each sensory improvement on the robot’s dexterity.

1.5. Document Structure

The remainder of the document is organized as follows:

- **Chapter 2** presents the state of the art. It reviews the background of the Vision-Language-Action paradigm, from imitation learning and Transformer-based policies to contemporary VLA models. Lines related to efficiency, memory, multi-modality, and three-dimensional perception are also analyzed.
- **Chapter 3** develops the theoretical foundations necessary to understand the work. These include imitation learning, the diffusion technique known as flow matching, temporal memory, and depth estimation.
- **Chapter 4** describes the materials and methods used. It presents the robotic platform, the data acquisition process, the hierarchical ablation study, the original model variants developed, the experimental configurations, the training procedure, and the evaluation protocol.
- **Chapter 5** presents and discusses the experimental results of both the layer study and the final evaluation. Subsequently, the four model variants are compared to test the working hypotheses.
- Finally, **Chapter 6** summarizes the main conclusions of the work, outlines its limitations, and proposes potential lines for future work.

Capítulo 2

State of the Art

This chapter reviews the main background and evolutionary lines of the VLA paradigm. The review starts from its methodological precursors and examines the transition towards generalist, open, and computationally efficient policies.

The selection of works pays special attention to proposals that expand the information available to the policy, such as tactile perception, the three-dimensional structure of the environment, and the memory of the model. With this, the aim is to delimit the gap addressed in this work and justify the choice of SmolVLA as the reference architecture.

2.1. Precursors of the Current Paradigm

Prior to the explicit formulation of VLA models, various works were already investigating how to relate instructions, visual observations, and robotic actions through learned models. Within this stage, two particularly relevant lines can be identified. The first studies the use of three-dimensional representations and Transformer architectures for language-conditioned manipulation. The second focuses on learning fine manipulation tasks from human demonstrations and on the joint prediction of action sequences. Although both lines address different problems, their contributions later converge in the development of policies capable of interpreting the environment, understanding the task, and generating actions in an integrated manner.

One of the fundamental difficulties of robotic manipulation consists of translating a general instruction, such as opening a drawer or placing an object in a specific position, into a physically executable action. To do this, the policy must identify the relevant elements of the scene, locate their position, and determine how the end effector must be positioned to interact with them. This relationship between perception, language, and action requires representing not only the appearance of objects but also their spatial arrangement.

PerAct constitutes a representative precursor of this line of research [1]. The model addresses manipulation tasks conditioned by linguistic instructions and uses visual observations with depth information to build a three-dimensional representation of the environment. Its approach is based on the premise that the prediction of six-degree-of-freedom actions is more suitable when the policy has an explicit description

of the scene’s geometry.

To achieve this, PerAct transforms observations obtained from one or several cameras into a three-dimensional grid made up of voxels. Each voxel represents a region of space and incorporates visual and geometric information. This representation allows the policy to determine where the robot should act on a spatial reconstruction of the environment, rather than inferring the position of objects solely from two-dimensional images. As shown in Figure 2.1, this formulation allows expressing the scene in a structured three-dimensional space and placing both the policy’s prediction and the reference expert action upon it.

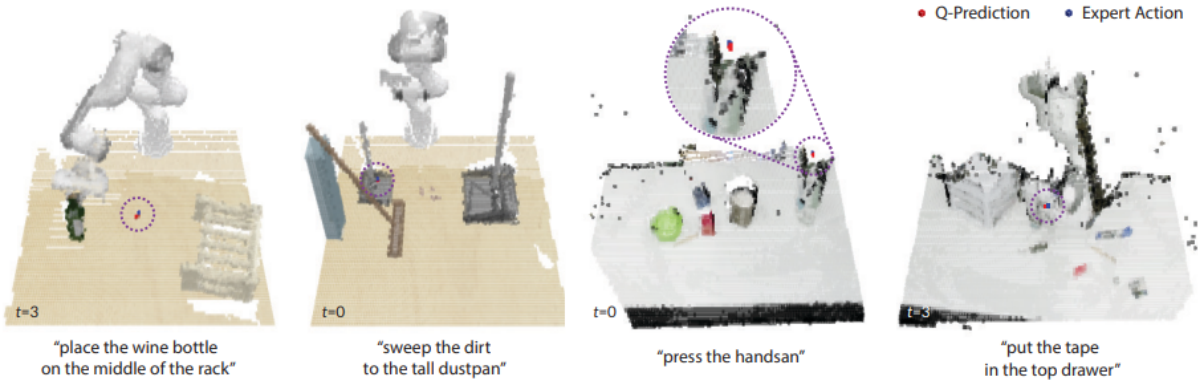


Figure 2.1: Qualitative examples of voxelized observations and action predictions in PerAct. The figure shows different manipulation scenes, along with the action predicted by the model and the reference expert action, both indicated on the three-dimensional representation of the environment. The examples on the left correspond to simulated tasks and those on the right to real tasks, adapted from [1].

The voxelized representation is processed by a Perceiver Transformer conditioned by the linguistic instruction. From it, the model predicts a discretized six-degree-of-freedom action, which includes position, orientation, and gripper state. This formulation is posed as the selection of the most suitable next action voxel, referred to in the work as the next best voxel action [1].

The use of a voxelized representation provides several advantages. First, it allows naturally fusing observations from different cameras. Second, it introduces explicit information about depth, position, and orientation, variables directly related to the execution of manipulation actions. Finally, it facilitates the application of spatial transformations during data augmentation, which improves the utilization of relatively small sets of demonstrations.

PerAct is not considered a VLA in the sense later adopted by models built on large pre-trained vision and language models. Its linguistic understanding remains closely tied to the tasks and demonstrations used during training, and the policy acts reactively based on the current observation, without maintaining an explicit record of history [1]. However, the work anticipates three elements that acquire central importance in subsequent developments: linguistic conditioning, the use of Transformers to relate perception and action, and the incorporation of explicit spatial structure in manipulation policies.

This last contribution is especially relevant for the three-dimensional line reviewed

later in this chapter. PerAct’s formulation shows that the representation used by the policy constitutes a fundamental part of the problem. When the task requires reasoning about geometry, depth, and orientation, providing an explicit three-dimensional structure can promote more efficient learning than the exclusive use of RGB images.

However, an adequate representation of the environment does not by itself solve another of the main challenges of robotic learning: the acquisition of precise behaviors from real data and their stable execution over time. Even when the policy has sufficient spatial information, fine manipulation tasks require high-quality demonstrations, continuous visual coordination, and mechanisms capable of limiting error accumulation between consecutive actions. This second line of background is represented by ALOHA and Action Chunking with Transformers (ACT) [2].

The proposal addresses bimanual fine manipulation tasks that demand continuous visual coordination, precise movements, and appropriate control of contacts between the robot and objects. In these tasks, a small error in the position of the end effectors can modify the state of the scene and condition subsequent decisions. As a result, the policy may receive observations increasingly distant from those used during training, which favors the progressive accumulation of errors.

To address this problem from the perspective of data acquisition, ALOHA provides a bimanual teleoperation platform built with low-cost components and designed to collect human demonstrations on physical robots. The operator controls two leader arms, whose movements are transferred to the follower arms responsible for executing the task. As shown in Figure 2.2, the visual and proprioceptive observations recorded during these demonstrations are subsequently used to train an end-to-end policy.

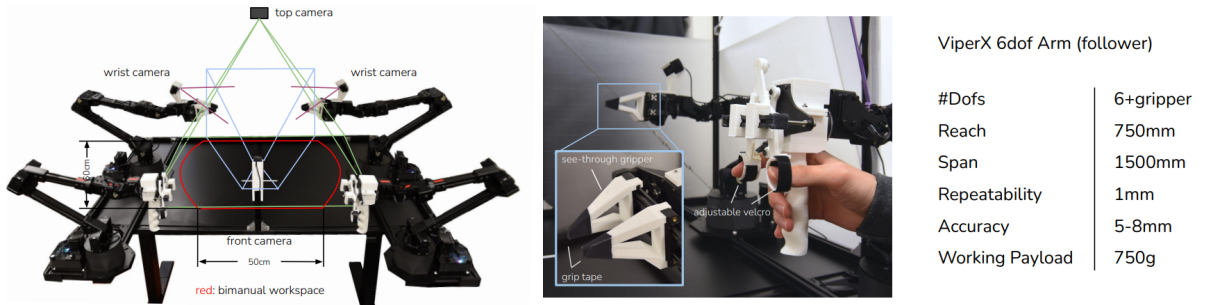


Figure 2.2: Imitation learning process used in ALOHA and ACT. Demonstrations are collected through a bimanual teleoperation system and used to train a policy that generates action sequences from visual and proprioceptive observations [2].

This infrastructure addresses one of the main constraints of robotic learning, since obtaining real data is costly and requires sufficiently precise teleoperation systems to capture complex behaviors. The proposal allows collecting demonstrations for tasks such as object insertion, container opening, and coordinated manipulation of deformable or small elements [2].

The availability of precise demonstrations allows addressing the second component of the problem: how to generate stable actions from them. ACT, a Transformer-based policy that seeks to reduce the error accumulation typical of imitation learning, is trained on these data. Policies that predict a single action at each time step must make a large number of consecutive decisions. Each error modifies the state observed

in the next step and can progressively shift the robot outside the distribution covered by the demonstrations.

ACT addresses this problem through action chunking. Instead of generating a single action, the policy predicts a block of future joint positions. This mechanism reduces the effective decision horizon and allows representing coordinated movements as a temporal unit. The architecture is trained as a conditional variational autoencoder, which allows modeling part of the variability present in human demonstrations [2].

However, the generation of complete blocks can produce discontinuities when replacing one sequence with the next. To reduce this effect, ACT incorporates a temporal ensembling mechanism. The policy is queried recurrently, so different blocks contain predictions for the same future time step. The executed action is obtained through a weighted combination of these overlapping predictions.

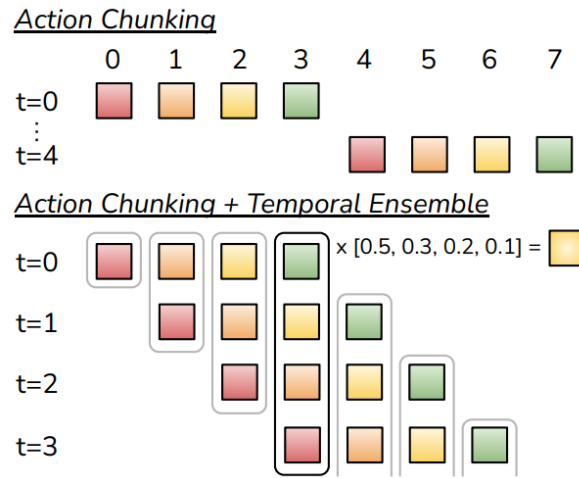


Figura 2.3: Operation of action chunking and temporal ensembling in ACT. The policy predicts blocks of k future actions and combines the overlapping predictions corresponding to a single time step to obtain the finally executed action [2].

Figure 2.3 represents both mechanisms. Block prediction reduces the effective length of the decision sequence, while temporal ensembling smooths the transition between consecutive predictions. This combination allows obtaining more stable trajectories and limiting error propagation during execution.

The contributions of PerAct, ALOHA, and ACT are complementary. PerAct focuses on how to represent the environment and spatially locate the action, while ALOHA and ACT address how to obtain precise demonstrations and turn them into stable control sequences. Together, these works anticipate several of the components that are later integrated into VLA policies: linguistic conditioning, the processing of observations using Transformers, the spatial representation of the environment, imitation learning, and the generation of action blocks.

In this way, PerAct demonstrates the utility of combining linguistic instructions, Transformer architectures, and three-dimensional spatial representations to predict manipulation actions. ALOHA reinforces the role of human demonstrations and facilitates their collection through an accessible robotic platform. ACT, on the other hand, shows that the joint generation of action sequences can reduce error accumulation in tasks requiring fine control.

These works establish three relevant foundations for the subsequent development of VLA models: the explicit spatial representation introduced by PerAct, the collection of demonstrations via ALOHA, and the stable generation of action sequences proposed by ACT. The subsequent combination of these ideas with pre-trained vision and language models allows moving towards more general, precise, and adaptable policies.

2.2. Birth and Consolidation of the VLA Paradigm

Building upon the background described above, Vision-Language-Action models introduce a significant shift in robotic learning by reusing vision and language models pre-trained on large collections of images and texts. In this way, policies can leverage previously acquired visual and semantic representations and adapt them to generate robotic actions, instead of relying exclusively on data specific to a task or platform.

The development of the paradigm progressively shifts towards the integration of perception, language, and control within the same architecture, as well as towards generalization across tasks and robots, the use of large demonstration datasets, the openness of models and training procedures, and the generation of continuous actions through specialized modules. This evolution lays the foundations for current VLA architectures.

The first step in this consolidation consists of incorporating robotic actions within the same generative interface used by vision and language models. RT-2 explicitly introduces the term Vision-Language-Action to describe models that process visual observations and linguistic instructions and directly generate robotic actions [3]. The proposal starts with a vision-language model (VLM) pre-trained with data from the Internet and adapts it for control through joint training on vision-language tasks and robotic trajectories.

Conventional vision-language models generate text sequences. However, a robotic policy must produce values that represent displacements, rotations, and commands to open or close the gripper. To make both outputs compatible, RT-2 applies action tokenization: each continuous dimension of the action is discretized and represented by tokens belonging to the model’s vocabulary.

In this way, the response to a robotic instruction formally adopts the same structure as a textual response. Given an image and an instruction, the model generates a sequence of tokens through autoregressive generation. Subsequently, these tokens are converted back into numerical values that can be sent to the robot controller.

Figure 2.4 summarizes this procedure. The main contribution of the architecture consists of integrating actions within the same sequential interface used to represent language. This unification allows jointly training examples of visual questions, linguistic instructions, and robotic control without introducing a completely independent action decoder.

This approach has two relevant consequences. First, it allows the direct reuse of architectures and infrastructures developed for large language and vision models. Second, it facilitates the transfer of semantic knowledge from web data to robotic tasks. RT-2 shows, for example, the ability to relate instructions to object properties,

2.2. Birth and Consolidation of the VLA Paradigm

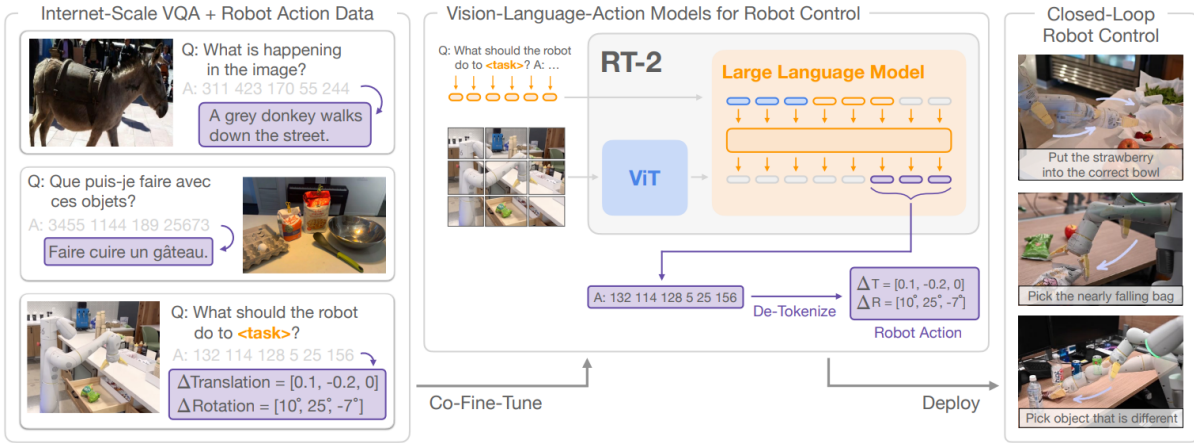


Figura 2.4: Conceptual diagram of RT-2. Continuous robotic actions are discretized and represented by tokens, which are generated autoregressively and subsequently converted into executable control commands [3].

interpret spatial relationships, and apply known manipulation skills to prompts that do not literally appear in the robotic demonstrations [3].

However, the discrete representation also introduces limitations. Action precision is constrained by the level of discretization, and autoregressive generation requires producing tokens sequentially. This process can increase latency and make it difficult to represent complex continuous distributions. These restrictions subsequently motivate the development of action decoders based on diffusion or flow matching.

Despite these limitations, RT-2 constitutes a turning point because it provides a clear formulation of the VLA paradigm. Visual perception, instruction understanding, and action generation are no longer treated as completely separate modules and are integrated within the same generative model.

Once the viability of this integration is demonstrated, the next challenge is to expand its scope beyond a specific robotic configuration. The consolidation of generalist robotic learning depends not only on combining vision, language, and action but also on the policy being reusable in applications with different sensors, instructions, robots, and action spaces. Octo addresses this problem through a generalist Transformer-based policy pre-trained on approximately 800,000 trajectories from Open X-Embodiment [4].

Octo does not use exactly the same formulation as RT-2. Instead of adapting a large VLM to generate action tokens, it builds a Transformer policy in which observations, task specification, and outputs are represented by different groups of tokens. The architecture accepts linguistic instructions or goal images and can process various combinations of cameras and proprioceptive states.

As depicted in Figure 2.5, the Transformer backbone receives the tokens available for each configuration and generates representations that connect to specific action heads. This organization allows modifying inputs or outputs without completely replacing the pre-trained model.

One of Octo's main contributions is its adaptability to new domains. By incorporating specific encoders or action heads, the policy can adjust to robots with observations,

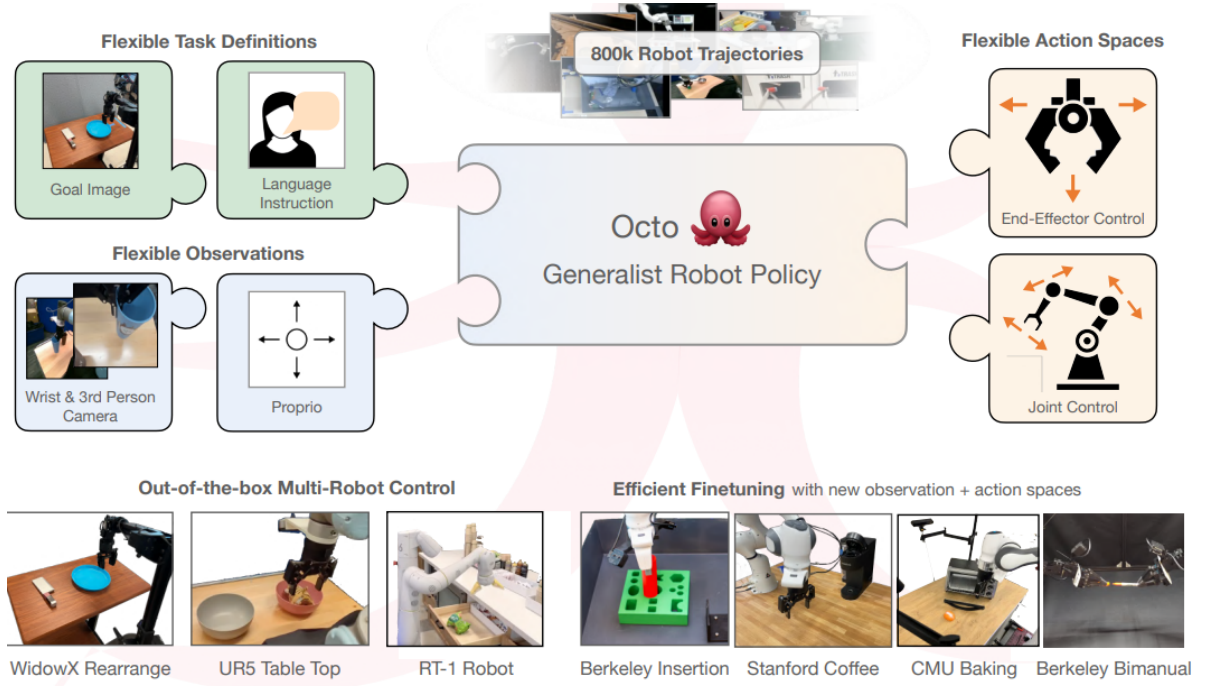


Figure 2.5: Conceptual representation of Octo. Instructions, visual and proprioceptive observations, and representations used to generate actions are integrated through a common Transformer architecture, adaptable to different robotic configurations [4].

morphologies, or control spaces different from those used during pre-training. This procedure reduces the need to train a complete policy from a random initialization.

Octo also demonstrates the importance of pre-training data diversity. Combining trajectories from different platforms allows learning shared representations across tasks and robots, even if the sensory configurations and action spaces are not identical. The work indicates that policies trained on broader data mixtures exhibit greater reusability capabilities than those developed on more restricted distributions [4].

Although Octo does not strictly correspond to the autoregressive formulation used by RT-2, it shares the central objective of VLA models: to develop a multimodal policy capable of interpreting different task specifications and generating actions for multiple robotic configurations. It also establishes properties that acquire increasing importance in subsequent works, such as adaptation to new observations, action spaces, and morphologies.

The evolution from RT-2 to Octo thus expands the paradigm from the transfer of vision-language knowledge towards robotic control to the pre-training of reusable policies across multiple platforms. However, both approaches leave an open fundamental question: how to represent and generate continuous actions with sufficient precision and speed. Models based on tokenized actions provide direct integration with language models, but their discrete output can limit control. π_0 addresses this difficulty through an architecture that combines a vision-language backbone with a specialized module for continuous action generation [5].

This module is called the action expert. The VLM processes the images and instruction to obtain a contextual representation of the task, while the expert uses this

2.2. Birth and Consolidation of the VLA Paradigm

information, along with the proprioceptive state and a sequence of noise-perturbed actions, to generate a continuous block of actions.

The general organization is presented in Figure 2.6. The vision-language backbone provides the representations that condition the cross-attention layers of the action expert. The inputs corresponding to state and actions are processed in a specialized flow, allowing the semantic knowledge of the VLM to be preserved without requiring the language model itself to directly generate every control dimension.

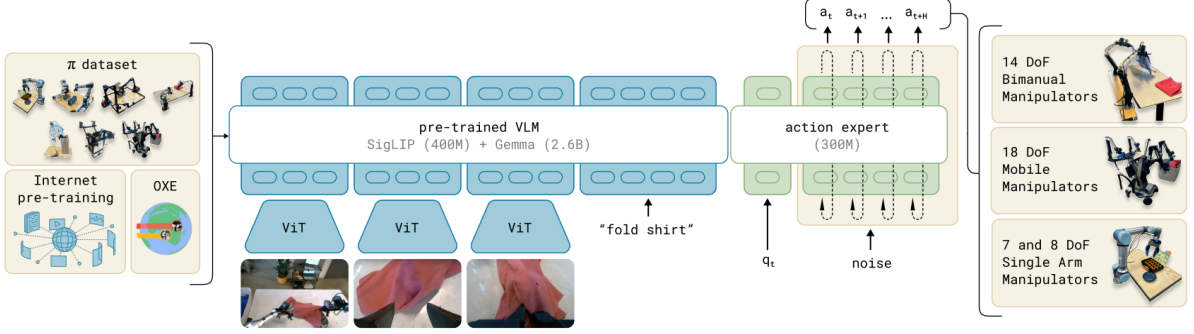


Figure 2.6: Conceptual diagram of Π_0 . The vision-language backbone processes images and instruction, while an action expert conditioned by its representations generates blocks of continuous actions [5].

The expert is trained using flow matching, a generative procedure that transforms an initial noise sequence into a continuous action block conditioned by observations, instruction, and robot state. This mechanism directly models continuous actions without resorting to discretization and is developed in greater detail in Section 3.2.

This formulation allows representing continuous and multimodal action distributions more flexibly than uniform discretization. It also facilitates the joint generation of temporal blocks, retrieving the concept of action chunking previously presented in ACT. The difference lies in the fact that the block is not obtained through a conditional variational autoencoder, but through a generative process conditioned by the representations of the vision-language model.

Π_0 is trained with data from various platforms, including single-arm robots, bimanual systems, and mobile manipulators. This diversity aims for the model to learn shared representations across tasks and morphologies, even though each platform has different state and action spaces. Subsequently, the model can be fine-tuned with task-specific demonstrations to improve precision and execution robustness.

The incorporation of the action expert modifies the relationship between multimodal understanding and robotic control. In RT-2, actions are integrated directly into the generative output of the vision and language model. In Π_0 , the VLM provides the semantic and visual context, while a specialized module models the continuous action distribution. This functional separation influences subsequent architectures, including GR00T N1 and SmoVLA, which also combine a vision and language module with a specific component for motion generation.

Overall, RT-2, Octo, and Π_0 represent different stages in the consolidation of generalist robotic policies. RT-2 establishes a common interface for vision, language, and tokenized actions. Octo demonstrates the viability of pre-training a flexible policy

with data from multiple robots. For its part, Π_0 introduces an action expert capable of generating continuous blocks via flow matching, partially separating multimodal understanding from motion generation.

Alongside this architectural evolution, OpenVLA constitutes a transversal milestone by shifting the paradigm towards an open, reproducible, and adaptable ecosystem [6]. The publication of its weights, training code, and fine-tuning tools facilitates the model’s reuse in new tasks and platforms, as well as providing a common foundation for further developments. Given that its main contribution relates to the openness, efficient adaptation, and accessibility of VLA models, these aspects are analyzed in greater detail in the section dedicated to open and efficient models.

Building on these contributions, research is no longer focused exclusively on demonstrating the viability of VLA models. Subsequent works study their application in more complex robotic platforms, the execution of prolonged tasks in unobserved environments, the reduction of computational cost, and the incorporation of new information sources, such as memory, touch, and three-dimensional geometry.

2.3. Generalization to New Environments and Platforms

Once the viability of Vision-Language-Action models has been demonstrated, research begins to study their application in more demanding conditions. Initial works show that these policies can follow instructions and execute manipulation tasks, but their performance still relies heavily on the proximity between training and deployment scenarios. The presence of new environments, different robotic platforms, or longer-duration tasks poses additional generalization difficulties.

Within this stage, two complementary directions can be distinguished. GR00T N1 extends the paradigm towards robotic platforms with more complex morphologies, especially humanoids, and proposes a training strategy based on heterogeneous data sources. On the other hand, $\Pi_{0.5}$ studies generalization in domestic environments not observed during training and combines high-level semantic prediction with continuous action generation. Both proposals show that expanding VLA models depends not only on scaling up the model size but also on adapting the architecture and data composition to more diverse execution contexts.

The first of these directions is represented by GR00T N1, an open foundational model geared towards the control of humanoid robots [7]. The transition from conventional robotic arms or bimanual tabletop systems to humanoid platforms increases problem complexity. These platforms feature a higher number of degrees of freedom, larger action spaces, and coordination requirements between different body parts.

To meet these demands, GR00T N1 deploys an architecture composed of two functional systems. The module referred to as System 2 uses a vision-language model to process visual observations and the instruction, interpret the environment, and obtain a semantic representation of the task. Subsequently, System 1 generates motor actions using a Diffusion Transformer trained with flow matching.

Both components are tightly coupled and optimized jointly. The information produced by the vision-language module directly conditions motion generation, avoiding a complete separation between reasoning and control. As shown in Figure 2.7, the

2.3. Generalization to New Environments and Platforms

model maintains distinct flows for multimodal interpretation and action production but integrates them within a single end-to-end training process.

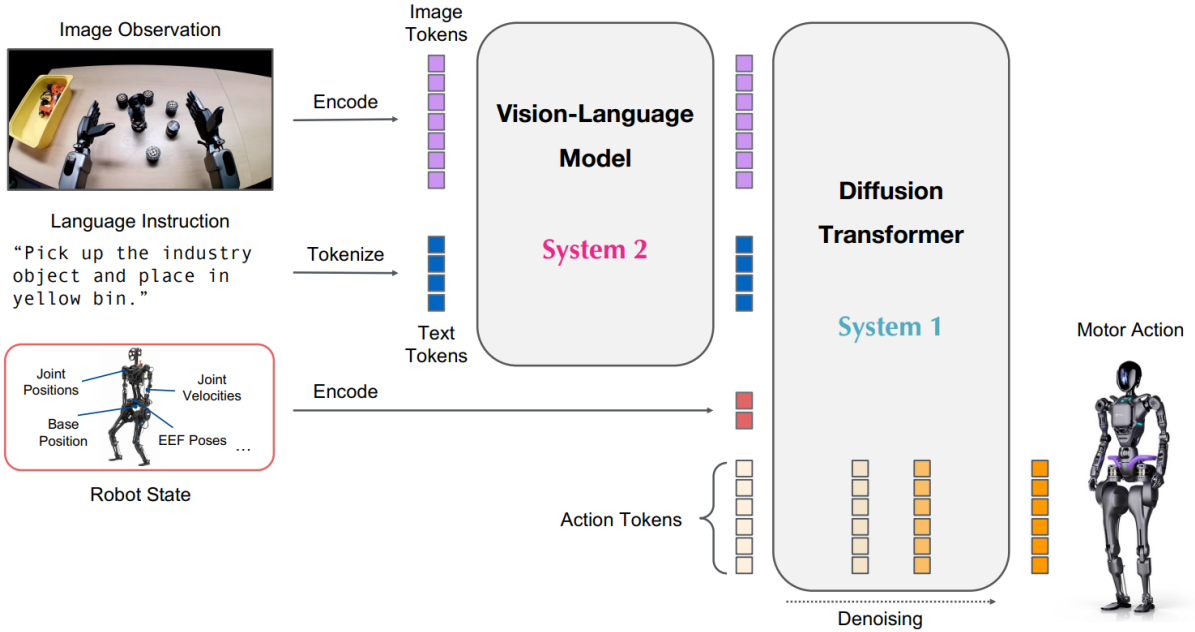


Figure 2.7: Dual architecture of GR00T N1. The vision-language module, named System 2, interprets observations and the instruction, while System 1 generates motor actions via a Diffusion Transformer conditioned by the preceding representations [7].

The second relevant contribution of the work lies in the organization of training data. Obtaining large quantities of real trajectories is particularly costly for humanoid robots, due to the difficulty of simultaneously teleoperating arms, hands, and other joints. GR00T N1 addresses this limitation through a combination of sources with different levels of scale and specificity.

This strategy is represented by a data pyramid. At its base lie large volumes of web data and human videos, providing general visual and behavioral knowledge. The intermediate level incorporates synthetic data, simulation trajectories, and sequences generated by other models. At the top are demonstrations collected directly on physical robots, which are lower in volume but closely related to actual execution.

Figure 2.8 summarizes this distribution. Moving up the pyramid decreases the amount of available information but increases its correspondence with the robot’s platform and action space.

The proposal also incorporates videos that do not contain explicit robotic actions. To use them during training, the work obtains latent actions or pseudo-actions through inverse dynamics models [7]. In this way, human or synthetic sequences can contribute information about visual changes and behaviors, even if they do not originally have associated joint commands.

This formulation broadens the concept of demonstration used in imitation learning. Training no longer relies exclusively on fully labeled robotic trajectories and shifts to integrate information from different domains. GR00T N1 thus shows that extending VLAs towards humanoid robots requires not only adapting the architecture but al-

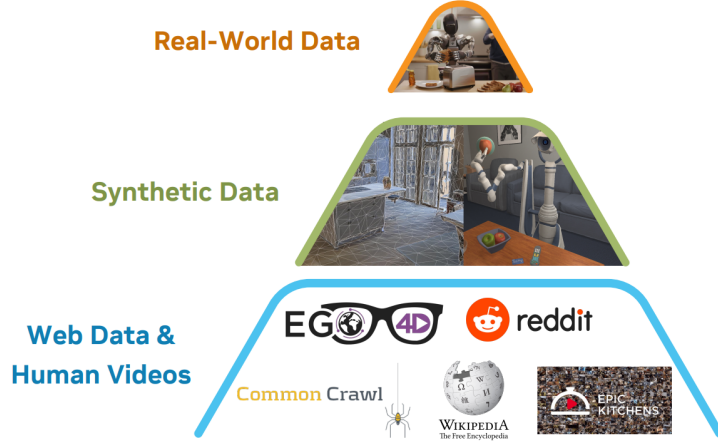


Figura 2.8: Data pyramid used in GROOT N1. Web data and human videos provide the largest scale base; synthetic and simulated data occupy the intermediate level; and physical robot trajectories constitute the most highly specific embodied source [7].

so designing procedures capable of leveraging data sources with varying degrees of relation to final control.

However, expanding the paradigm to new morphologies does not by itself solve the generalization problem. Even when a policy can control more complex platforms and leverage heterogeneous data sources, it remains necessary to verify whether that knowledge allows acting in scenarios different from those observed during training. This issue is especially relevant in domestic environments, where the distribution of objects, visual conditions, and required action sequences can vary considerably between homes.

In this context, $\pi_{0.5}$ focuses on generalization in domestic environments not included in the training data [8]. The objective is to control a mobile manipulator during prolonged tasks in homes with layouts, objects, and visual conditions different from those used during learning.

The proposal combines data from multiple robots, perception tasks, semantic annotations, verbal instructions, and multimodal examples obtained from the web. The subset most directly related to the final application contains approximately 400 hours of mobile manipulation in homes. However, during the first training phase, 97.6 % of the examples come from other sources [8].

This composition responds to the difficulty of collecting enough trajectories to cover all configurations that might appear in a home. Instead of trying to represent the variability of the real world exclusively through target robot demonstrations, the model transfers knowledge from related tasks and domains. Thus, $\pi_{0.5}$ shares with GROOT N1 the idea that specific robotic data must be complemented with broader sources, although it directs this strategy towards transfer between scenarios rather than adaptation to a specific morphology.

Training is organized into two stages. During the first, $\pi_{0.5}$ uses an autoregressive formulation where actions are represented by discretized tokens via FAST tokenization. In this phase, action prediction examples are combined with tasks such as

2.3. Generalization to New Environments and Platforms

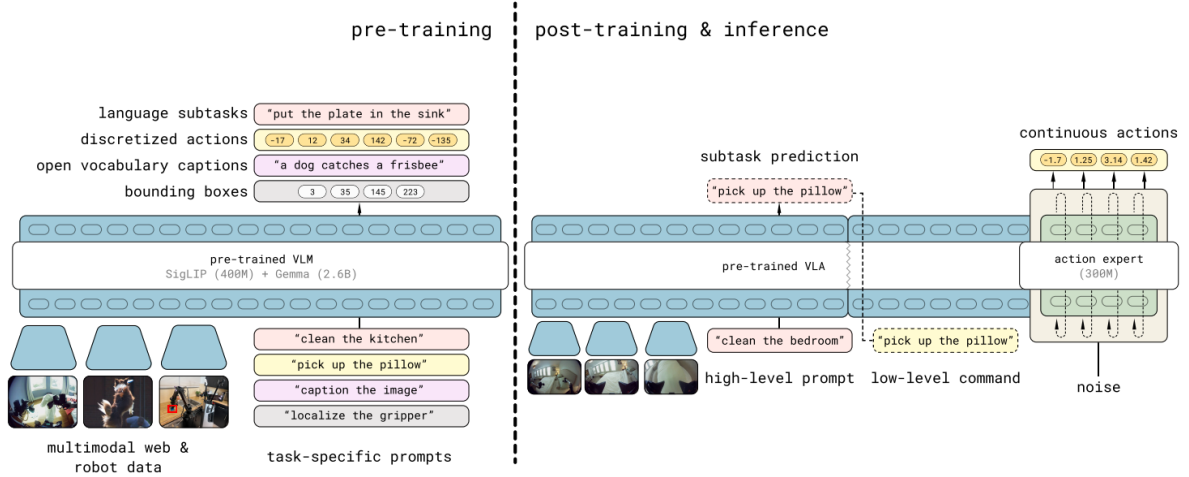


Figure 2.9: Training and inference process of $\Pi_{0.5}$. Pre-training integrates heterogeneous sources using discretized actions and multimodal tasks; subsequently, a continuous action expert based on flow matching generates action blocks for robotic control [8].

semantic subtask generation, bounding box grounding, image captioning, and visual question-answering.

This interface allows treating different supervision sources as multimodal sequences within the same training procedure. Perceptual and semantic tasks contribute to environment interpretation, while robotic trajectories provide the relationship between observations and actions.

In a second stage, the model incorporates the continuous action expert used in Π_0 . This component is trained via flow matching to generate continuous action blocks. The complete sequence is depicted in Figure 2.9: pre-training allows integrating heterogeneous data through a discrete interface, while post-training adapts the policy to the demands of continuous control.

The combination of both stages allows leveraging semantic knowledge and motor capabilities differentially. During inference, this separation is maintained through a hierarchical organization. The model first predicts a high-level semantic subtask, such as selecting or moving a specific object. Then, the action expert generates low-level commands conditioned by that subtask.

This division allows differential exploitation of the training sources. Web data, visual annotations, and verbal instructions reinforce semantic interpretation and subtask selection. Demonstrations from different robots, meanwhile, contribute to learning manipulation skills and movement sequences.

Figure 2.10 shows the main sources used by the model and their relationship with the two levels of the policy.

The work shows that this strategy allows executing tasks of between ten and fifteen minutes, such as cleaning kitchens and bedrooms, in homes that do not appear during training [8]. The main contribution lies not only in the duration of these executions but in demonstrating that data directly related to the final platform can be complemented with information from other robots and multimodal tasks.

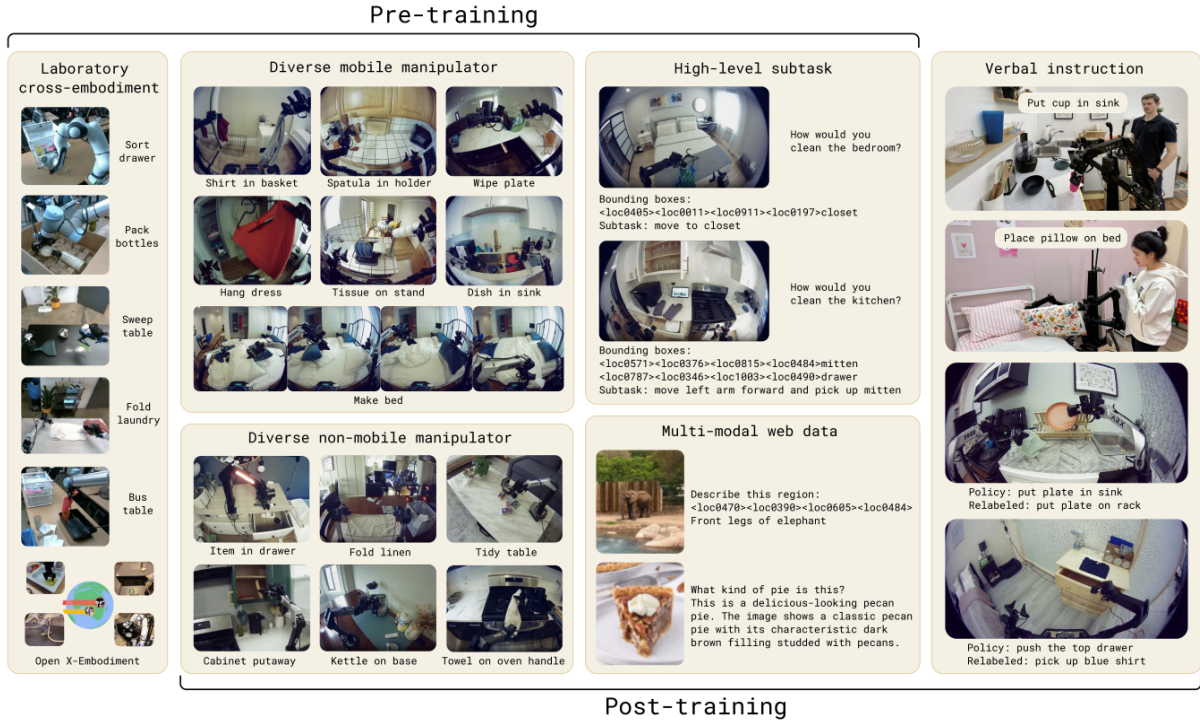


Figure 2.10: Heterogeneous sources used in $\Pi_{0.5}$. The model combines robotic trajectories, subtask prediction, verbal instructions, object detection, and multimodal examples from the web to favor transfer to unobserved domestic environments [8].

GR00T N1 and $\Pi_{0.5}$ thus represent two complementary ways to expand the VLA paradigm. GR00T N1 adapts architecture and data for humanoid platform control, where morphology and action space complexity increase. $\Pi_{0.5}$ uses heterogeneous co-training and a hierarchical policy to improve transfer to unobserved domestic environments and longer-duration tasks.

In both cases, generalization capacity depends not only on model size but also on data composition and how semantic understanding and motor control relate. This evolution shifts the focus from executing tasks in known configurations to policies capable of acting on increasingly diverse platforms and scenarios.

2.4. Open Models and Efficiency

The consolidation of a robotic learning paradigm requires that its models can be studied, reproduced, and adapted to new platforms. In the case of VLAs, this need takes on special importance due to the cost associated with large multimodal models, collecting robotic demonstrations, and training on specialized infrastructures.

From this need arises a line aimed at building a more open and accessible ecosystem. Proposals in this area address the publication of weights and code, efficient fine-tuning, reducing architecture sizes, and using lower-cost robotic platforms. These advances expand participation in the development of VLA models and facilitate their application outside heavily resourced industrial settings.

As mentioned in Section 2.2, OpenVLA shifts the VLA formulation towards an open and reproducible environment [6]. The model contains approximately 7 billion parameters and is trained with 970,000 Open X-Embodiment episodes.

The architecture combines two visual encoders with complementary functions. DINOv2 provides features related to the spatial structure of images, while SigLIP provides representations with higher semantic alignment to language. Outputs from both encoders are fused and projected into a Llama 2 backbone input space.

The instruction and visual features are processed jointly to generate action tokens. As in RT-2, continuous control dimensions are discretized, associated with vocabulary elements, and produced autoregressively. Finally, tokens are converted back into displacements, rotations, and gripper commands.

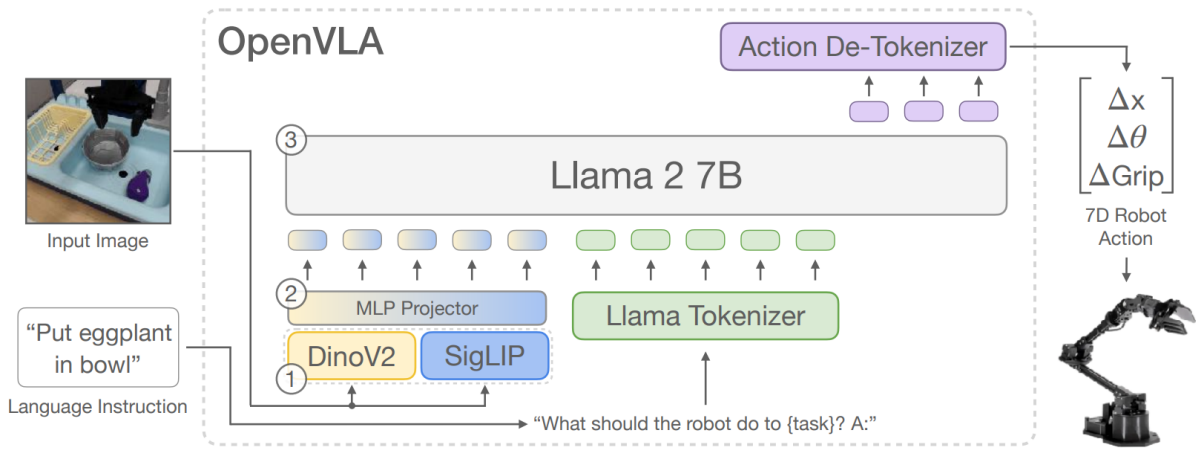


Figure 2.11: OpenVLA architecture. Visual representations extracted via DINOv2 and SigLIP are projected into the Llama 2 space, which generates discretized tokens later converted into continuous control commands [6].

OpenVLA’s main contribution within this line consists of publishing weights, training code, and necessary tools to adapt the model. This availability allows examining architecture composition, modifying learning procedures, and comparing new proposals on a shared foundation.

The work also studies model adaptation via Low-Rank Adaptation (LoRA). This technique introduces trainable low-rank matrices into certain network components while keeping most original parameters frozen. According to the results presented in the work, this procedure allows approximating full fine-tuning performance by updating only a small fraction of the model [6].

The use of 4-bit quantization during inference is also analyzed. Reducing numerical precision decreases memory consumption and facilitates deployment on lower-capacity hardware without causing a significant performance degradation as described in the study [6].

OpenVLA thus makes accessibility a central dimension of VLA development. Publishing the complete process and evaluating efficient fine-tuning techniques allow the model to be used as a starting point for new tasks without repeating the cost of its generalist pre-training.

However, opening the model and reducing the number of parameters updated during adaptation do not eliminate the cost associated with a multi-billion parameter architecture. Efficient fine-tuning facilitates OpenVLA’s reuse, but inference remains conditioned by backbone size and autoregressive action generation. The next evolution within this line is, therefore, to address efficiency from the architectural design itself.

TinyVLA targets these two limitations: the high inference cost of large VLAs and the amount of robotic data needed for their training [9]. The proposal replaces multi-billion parameter backbones with compact vision-language models of fewer than 1 billion parameters.

In addition to reducing the multimodal component size, TinyVLA modifies how actions are generated. Instead of using an autoregressive output based on discretized tokens, it incorporates a diffusion-based policy decoder. This module produces continuous action sequences and avoids separately generating each control component.

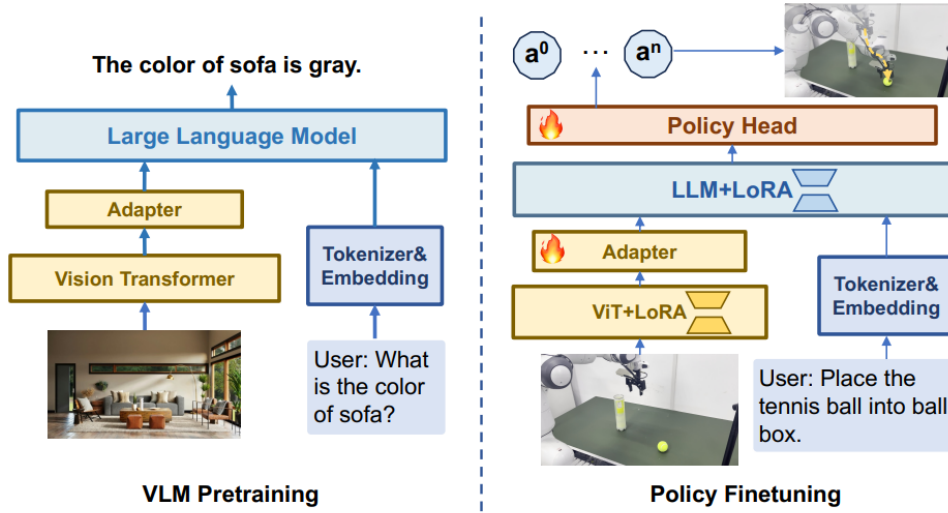


Figure 2.12: TinyVLA architecture and adaptation process. A compact vision-language model is combined with a diffusion policy that generates continuous action sequences, avoiding autoregressive prediction of control tokens [9].

This formulation seeks to retain representations learned during vision-language pre-training without relying on additional pre-training on massive robotic datasets. The policy is directly adapted to demonstrations corresponding to the target tasks.

Results described in the work show speed and data efficiency improvements over OpenVLA, alongside comparable or superior performance in different simulation and real robot configurations [9]. Bimanual tasks are also studied, where using a compact model and a continuous decoder avoids relying on the robotic pre-training distribution containing the exact same morphology.

TinyVLA shows that scaling down does not have to be limited to compressing an existing model. Architecture, training objective, and action representation can be jointly modified to reduce latency and improve the utilization of a limited demonstration set.

The transition from OpenVLA to TinyVLA reflects two complementary levels of accessibility. OpenVLA provides an open foundation and efficient fine-tuning procedu-

res. TinyVLA reduces size and replaces autoregressive generation with a continuous diffusion-based policy. Building on this evolution, SmoIVLA is developed, deepening efficiency and adding an explicit orientation toward community data, affordable platforms, and control continuity during inference.

2.4.1. SmoIVLA: Affordable Robotics

SmoIVLA continues the line of compact models but also incorporates an explicit orientation towards affordable robotic platforms and community-contributed datasets [10]. The model combines a small-sized VLM with an action expert trained via flow matching.

The vision-language component processes the instruction and images, while the expert receives its representations alongside robot state and a noisy action sequence. The expert alternates self-attention and cross-attention layers to generate a continuous action block.

The architecture introduces several decisions aimed at reducing computational cost. These include discarding the VLM’s final layers, using a reduced number of visual tokens, and employing lighter cross-attention layers within the action expert.

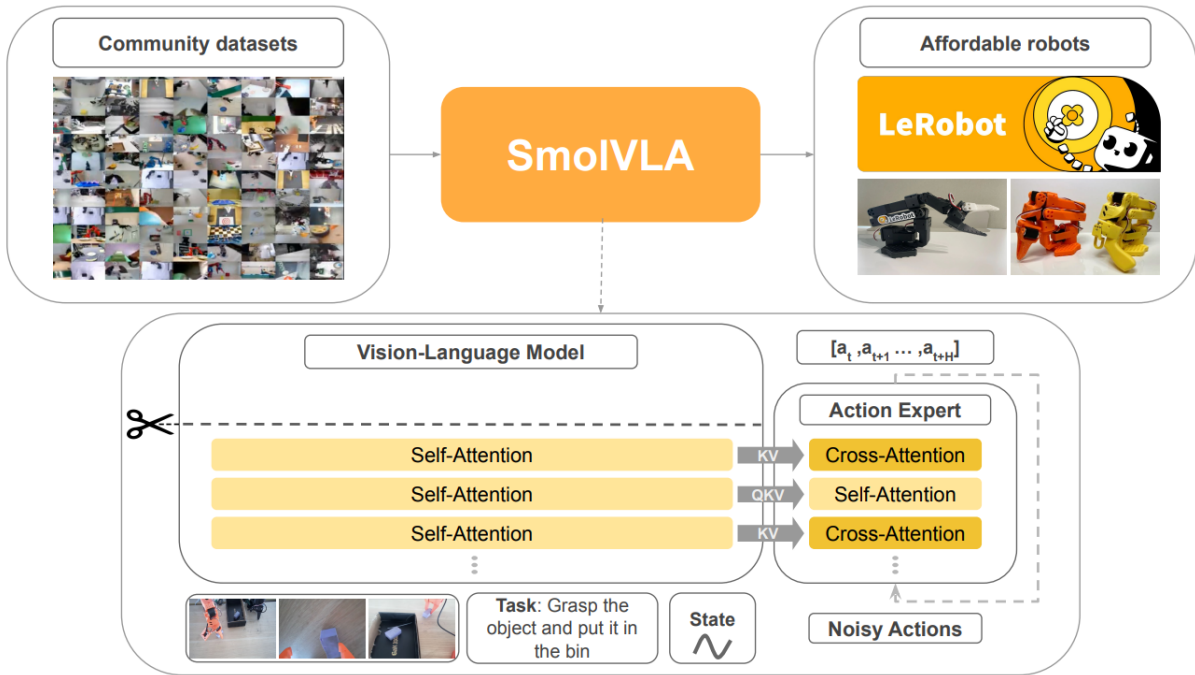


Figura 2.13: General scheme of SmoIVLA. The model combines a compact VLM with an action expert trained via flow matching and uses public datasets collected on affordable robotic platforms [10].

SmoIVLA also modifies policy execution through an asynchronous inference pipeline. In a synchronous scheme, the robot must wait for observation capture, processing, and next block generation to finish before continuing execution. This dependency can introduce pauses when inference time is high.

Asynchronous inference decouples the production of new action blocks from current

block execution. While the robot consumes already available actions, the model processes recent observations and prepares the next sequence. Figure 2.14 depicts this overlap between prediction and control.

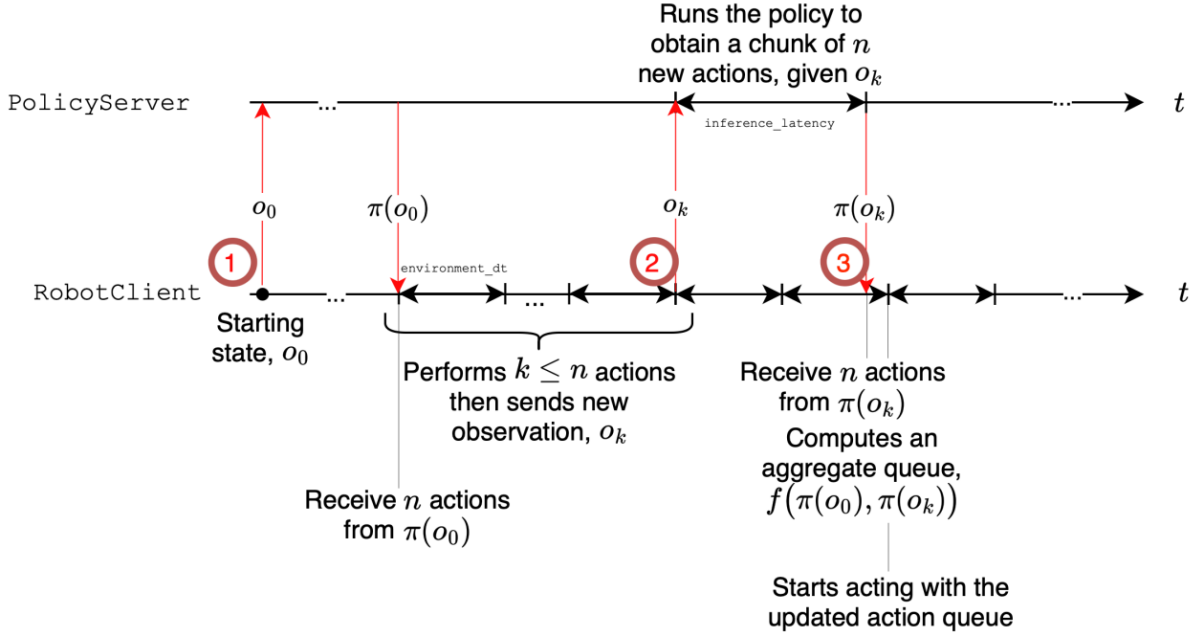


Figure 2.14: SmolVLA’s asynchronous inference pipeline. The generation of new action blocks overlaps with the execution of available blocks, reducing waiting periods between perception, prediction, and control [10].

This decoupling occurs in the execution flow and does not imply that modalities are processed independently within the model. The action expert remains conditioned by representations obtained from the VLM. The difference lies in the fact that prediction and actuation do not have to strictly finish sequentially at every time step.

This feature contrasts with the coupling used in GR00T N1. In both models, vision-language information directly conditions action generation. However, SmolVLA introduces a temporal separation during inference to improve control continuity, whereas GR00T N1 focuses on the joint integration of reasoning and actuation in humanoid platforms [7], [10].

OpenVLA, TinyVLA, and SmolVLA represent three complementary accessibility levels. OpenVLA provides an open base and efficient fine-tuning procedures. TinyVLA reduces size and replaces autoregressive generation with a continuous diffusion-based policy. SmolVLA combines a compact architecture, community data, and asynchronous inference to facilitate deployment on more limited platforms and resources.

This evolution shows that VLA development depends not solely on increasing parameter counts or data volume. Code openness, adaptability, inference latency, and data collection platform availability also constitute determining factors for practical use.

2.5. Tactile Perception, Memory, and Complex Tasks

The consolidation of VLA models allows shifting some research from basic vision, language, and action integration towards the acquisition of broader perceptual and temporal capabilities. In real tasks, a single visual observation can be insufficient to describe the environment’s state, especially when there are difficult-to-interpret contacts, occlusions, or dependencies between different execution stages.

For this reason, recent architectures incorporate additional modalities and mechanisms capable of preserving information over time. These extensions allow representing both physical interaction aspects and events that are no longer visible, providing the policy with richer context to select actions. Within this line, VTLA extends instantaneous perception via tactile information, while MEM incorporates memory to preserve relevant context from previous observations and actions.

Most previously reviewed VLA models use RGB images as their primary source of environment perception. This information allows recognizing objects, interpreting instructions, and estimating visible spatial relationships. However, in some tasks, the relevant interaction state also depends on forces, contacts, or minor misalignments that cannot be inferred accurately enough from an image.

This limitation is particularly critical in insertion operations. When an object makes contact with a surface or hole edges, small spatial differences can determine whether an action progresses correctly or produces a collision. In these situations, tactile information provides local contact signals that complement global visual perception.

VTLA expands the paradigm through a Vision-Tactile-Language-Action model designed for precise insertion tasks, where a part must be aligned and inserted into a geometrically matching cavity [11]. The policy processes a linguistic instruction, a wrist-camera image, and tactile image sequences captured by sensors located on the gripper fingers.

Figure 2.15 presents the proposal’s general organization. Visual and tactile modalities are integrated within a vision-language model, which generates an action aimed at correcting misalignment between the part and the hole.

Integrating tactile observations presents a temporal challenge. While a visual image provides a global description of the current state, tactile information is obtained as a short sequence whose changes reflect contact evolution. A conventional vision-language model can process individual images but is not necessarily prepared to represent the temporal dependencies of such signals.

To address this problem, VTLA introduces Vision-Guided Temporally Enhanced Tokens (VGTE). This mechanism transforms tactile sequences into temporally aware representations before incorporating them into the model. Visual information acts as a global reference, while tactile tokens provide local details about the contact and its variation during insertion attempts [11].

Training is organized into two phases. In the first, the model is fine-tuned via next-token prediction on a dataset composed of visual observations, tactile observations, instructions, and actions. Continuous actions are represented by tokens, maintaining a formulation similar to that used by autoregressive VLAs.

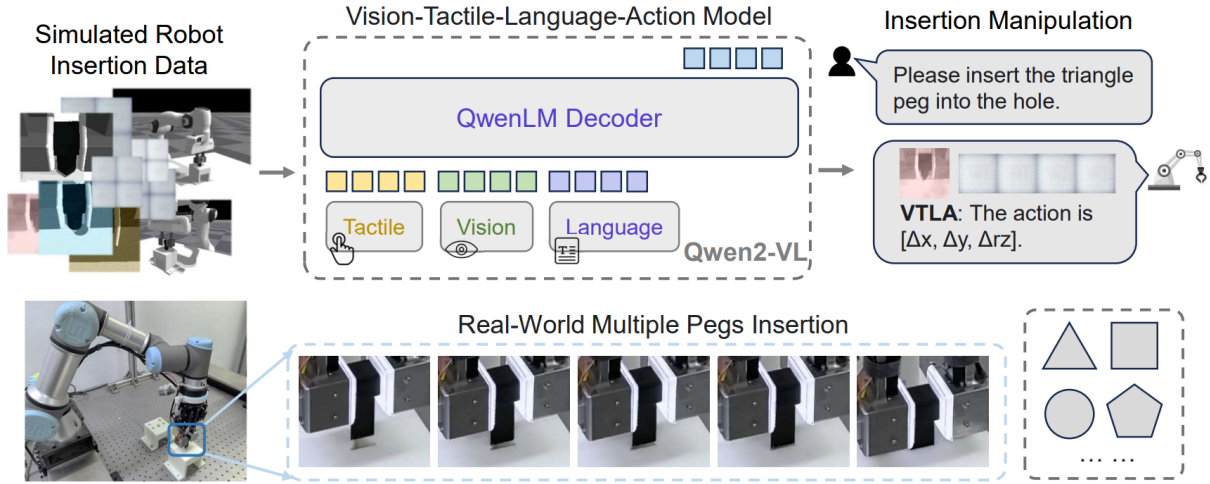


Figure 2.15: Overview of VTLA. The model processes visual and tactile observations along with a linguistic instruction to generate actions in peg-in-hole insertion tasks [11].

This representation introduces a discrepancy between the training objective and the nature of the problem. Next-token prediction is posed as a classification task over discrete elements, whereas robotic control requires approximating continuous values. To reduce this gap, the second phase incorporates Direct Preference Optimization (DPO).

The procedure generates multiple action predictions for the same observation and ranks them according to their proximity to the reference action. Closer actions are considered preferred, and those further away are used as rejected examples. Thus, the model receives an additional signal that approximates the behavior of regression supervision without completely abandoning its generative formulation [11].

Figure 2.16 summarizes both phases. Supervised fine-tuning initially establishes the relationship between modalities and actions, while DPO later refines predictions using preferred and rejected action pairs.

The training set is obtained in simulation via insertion tasks with different geometric shapes and clearance levels. Subsequently, the model is evaluated on a physical system without retraining with real data. Results presented show success rates exceeding 90% on shapes not observed during training and effective transfer from simulation to the real environment [11].

Comparisons also indicate that combining vision and touch outperforms variants using only one modality. Vision provides global information on part position and geometry, while touch enables interpreting the local contact state. VTLA’s contribution, therefore, lies in demonstrating that VLA multimodality can extend beyond image and language when a task demands physical information not fully represented in visual data.

Incorporating tactile signals expands the available information about the current interaction. However, having more instantaneous modalities does not solve tasks where part of the necessary context belongs to the past. A policy might accurately interpret present contact yet lose track of temporarily hidden objects, previously performed

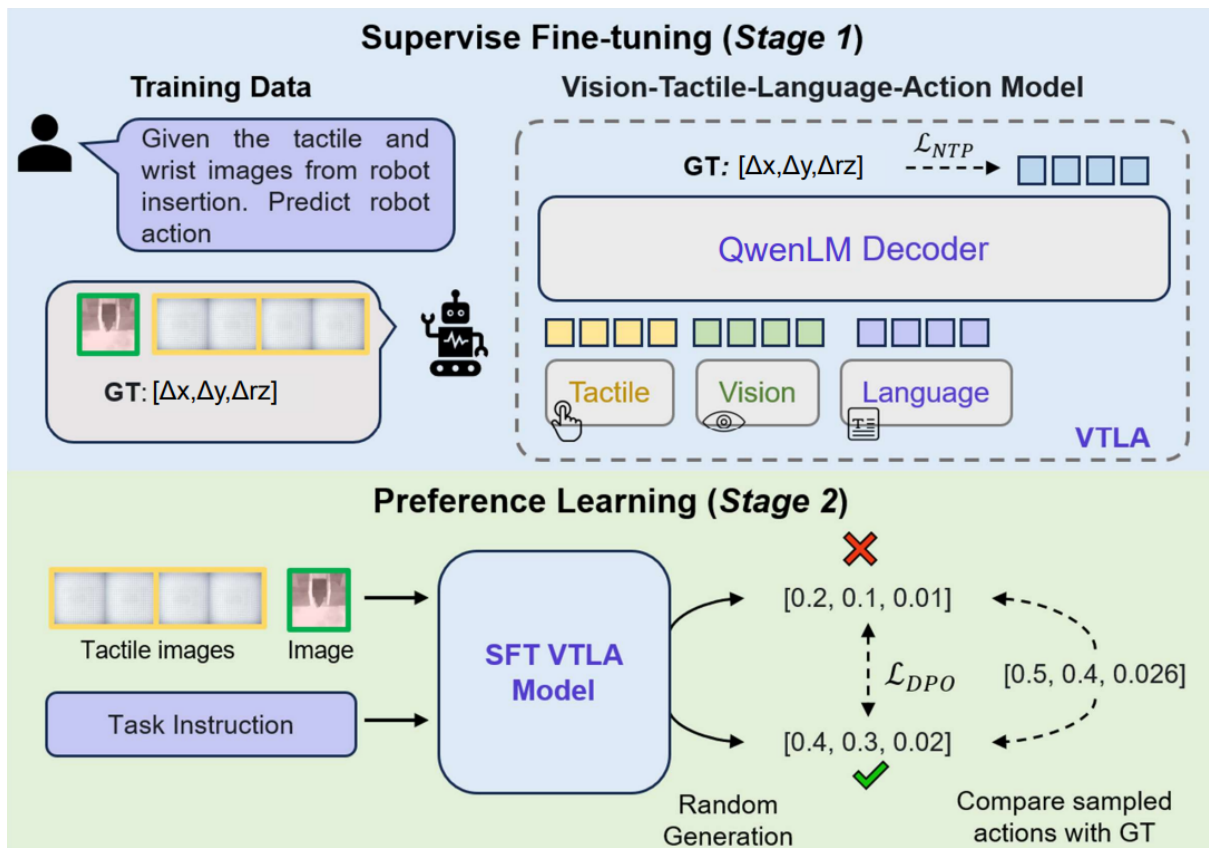


Figura 2.16: VTLA training pipeline. In the first phase, the model is fine-tuned via next-token prediction. In the second, DPO introduces a preference-based signal to better approximate the continuous nature of robotic actions [11].

actions, or stages completed during prolonged execution.

Many robotic policies act solely based on current observations or a short time window. This formulation proves insufficient when an object is occluded, an earlier action modifies task state, or execution requires remembering which stages are already complete.

One possible solution is feeding all past observations into the policy's context. However, this increases computational cost as execution progresses. Furthermore, not all past information requires the same detail level or needs to be retained for the same duration.

MEM builds upon this difference and proposes a multi-scale memory architecture for VLA models [12]. The proposal distinguishes between recent visual information, necessary to interpret motion and occlusions, and longer-duration semantic information, used to represent overall task progress.

The first component is a short-term video memory. This memory retains a dense sequence of recent observations and compresses it via an efficient video encoder. Its objective is to preserve visual and temporal details that might be necessary to interpret scene dynamics.

This representation allows, for instance, remembering an object's position when temporarily occluded by the robot arm, detecting an incomplete grasp, or modifying a manipulation strategy after a failed attempt. By using a sequence of nearby images, the policy accesses recent change information and is not solely reliant on an isolated image.

The second component is a long-term text memory. Instead of retaining images throughout execution, the system maintains a linguistic summary of past events that remain relevant. This representation can indicate which ingredients have already been used, which zones have been cleaned, or which subtasks are pending [12].

Figure 2.17 shows this separation. Visual memory preserves dense information over a short horizon, while text memory compresses task progress into semantic concepts maintainable over longer intervals.

This organization avoids applying a single representation to all time horizons. Images preserve spatial and dynamic detail, but their storage and processing are costly. Language provides a more compact representation, though it lacks precise geometry or motion retention. MEM assigns each format to the information type for which it is best suited.

The architecture functionally divides into a high-level and a low-level policy. The high-level policy updates linguistic memory and generates the next subtask to execute. The low-level policy receives that subtask, current observations, recent visual memory, and the textual summary to produce continuous actions via an action expert.

Figure 2.18 depicts this relationship. The upper level maintains semantic task state, while the lower level uses more detailed visual info to control the robot. Both memory scales combine during action generation but fulfill different system roles.

The video encoder is necessary to keep latency within limits required by robotic control. Processing each historical image independently via the full VLA backbone would

2.5. Tactile Perception, Memory, and Complex Tasks

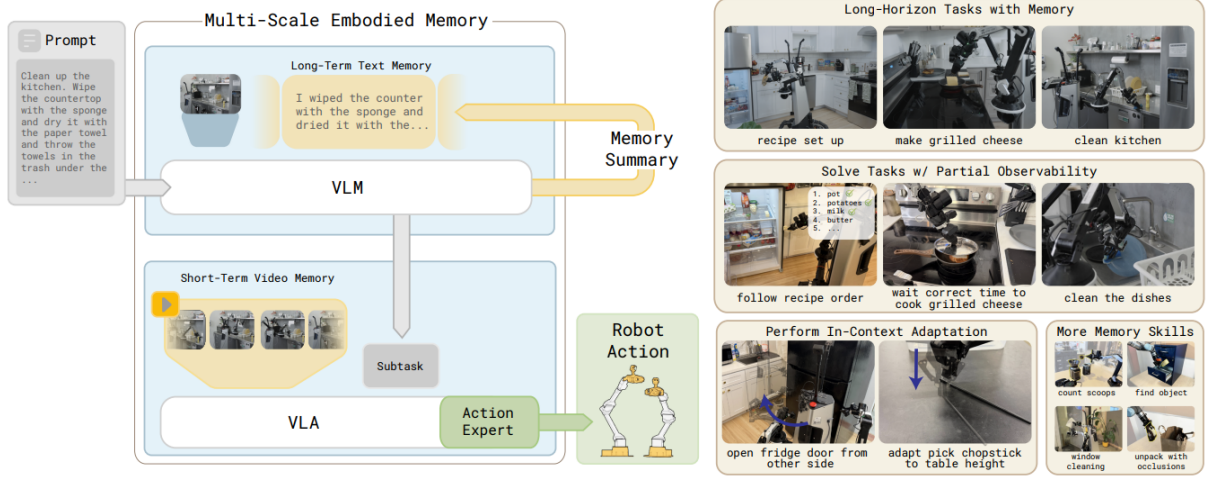


Figure 2.17: Overview of MEM. The system combines a short-term visual memory, based on video sequences, with a long-term text memory that summarizes semantic events relevant to the task [12].

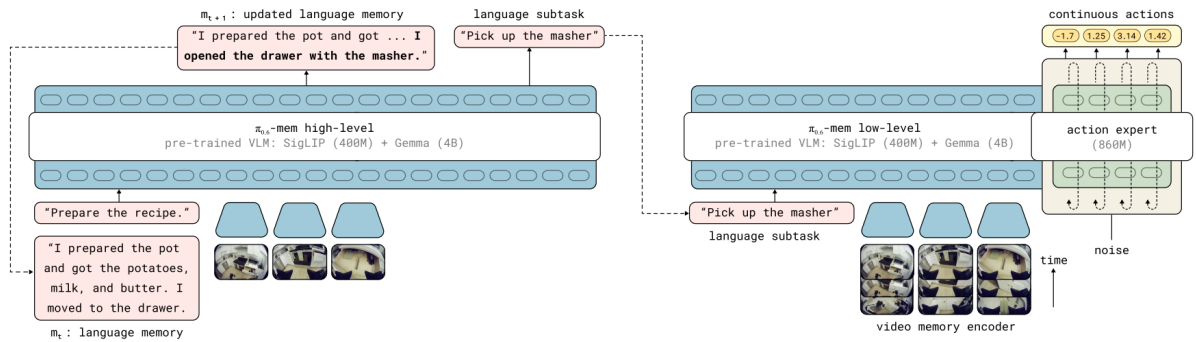


Figure 2.18: Functional architecture of MEM. The high-level policy updates text memory and selects subtasks, while the low-level policy uses recent observations, visual memory, and semantic context to generate continuous actions [12].

progressively increase inference time. Temporal compression allows using several recent observations without treating them all as independent visual inputs [12].

Results show MEM can retain information during tasks lasting up to fifteen minutes. Evaluations include prolonged activities, like cleaning a kitchen or preparing a recipe, where the policy must coordinate multiple stages and remember events no longer visible in the current observation [12].

Short-term memory also improves response to partial observability and self-occlusions. It also enables in-context adaptation of manipulation strategies: after observing a failure, the policy can use the recent sequence to modify its next attempt without updating its parameters.

MEM demonstrates that adding memory to a VLA isn't just about expanding its input window. The policy must select what information to keep, for how long, and via what representation. The separation between visual and text memory allows combining local precision and semantic compression without requiring the entire history to be processed at the same detail level.

VTLA and MEM expand the paradigm in different directions but respond to a common limitation: the current visual observation does not always contain all information needed to act. VTLA incorporates tactile signals to represent the local physical state of an interaction. MEM preserves past information to resolve occlusions, adapt strategies, and maintain progress over extended tasks.

These proposals show VLA evolution isn't limited to scaling up. A more general policy must also integrate modalities suited to the interaction type and maintain temporal representations compatible with task duration. The following research line addresses another control information limitation: the lack of explicit 3D geometric representation in models built primarily on 2D observations.

2.6. Three-Dimensional Geometric Representation

Besides expanding perceptual modalities and temporal context, robotic policies must accurately represent the environment's spatial structure. RGB images provide relevant visual and semantic information but reduce a 3D scene to a 2D projection. This information loss can cause ambiguities in estimating depth, height, orientation, or inter-object distance, aspects directly related to executing movements in manipulation tasks.

Incorporating 3D structure into robotic learning predates VLA models. As described in Section 2.1, PerAct uses RGB-D observations to build a voxelized environment representation and predict discretized 6-DoF actions [1]. This formulation shows providing explicit geometric representation can facilitate the perception-action relationship. Building on this, subsequent research studies how to transfer 3D info into generative policies and pre-trained VLA models, aiming to preserve previously acquired visual and semantic representations.

A first step is verifying if 3D geometry can also improve policies that generate continuous action sequences. Visual policies based solely on 2D images may need many demonstrations and struggle to generalize when object position, viewpoint, or scene

2.6. Three-Dimensional Geometric Representation

appearance change. DP3 (3D Diffusion Policy) addresses this limitation via a diffusion policy conditioned by a compact 3D representation [13].

The model takes depth information and transforms it into a point cloud. This representation preserves the 3D position of a set of observed scene points, providing explicit info on geometry and object spatial distribution. Before conditioning the policy, the cloud is cropped and downsampled to reduce size.

Next, a lightweight encoder transforms the point cloud into a lower-dimensional latent representation. This representation combines with robot state and conditions a diffusion policy tasked with generating continuous action sequences.

The architecture is summarized in Figure 2.19. Depth isn't just processed as an extra image channel; it becomes a 3D structure from which a compact scene description is derived. This description subsequently guides motion generation.

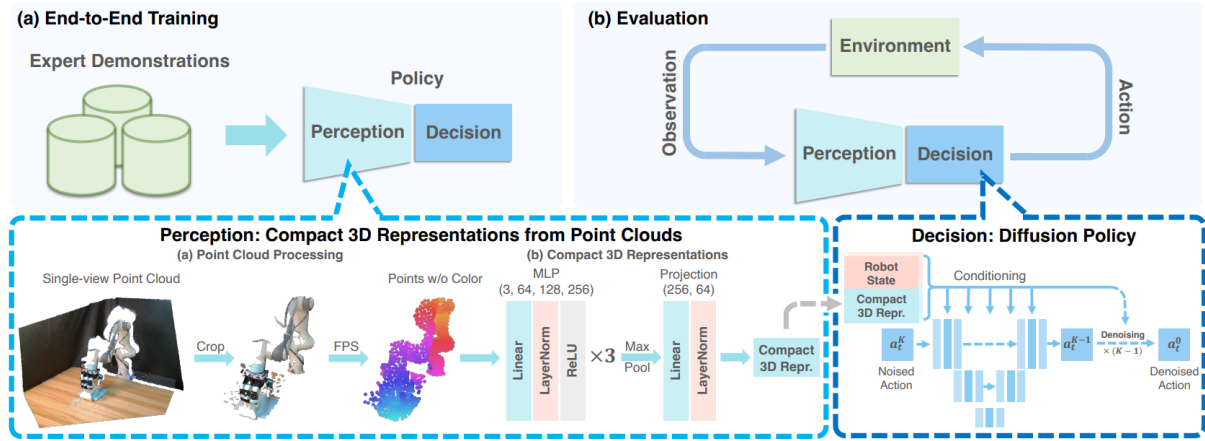


Figure 2.19: 3D Diffusion Policy architecture. Depth info transforms into a point cloud, gets processed by a lightweight encoder, and conditions a diffusion policy responsible for generating continuous action sequences [13].

DP3 verifies that 3D structure benefits don't depend on a single architectural family. In PerAct, 3D representation conditions a Transformer selecting discretized actions. In DP3, the same idea applies to a generative policy directly modeling continuous control sequences.

Work results indicate precision, data efficiency, and generalization improvements over 2D visual policies [13]. Evaluations include simultaneous changes in object position, viewpoint, appearance, and scene instances.

Ablation studies also analyze different ways to incorporate spatial info. In DP3's experimental setup, point cloud representation outperforms alternatives based directly on depth, RGB-D images, or voxels [13]. Likewise, a compact encoder can provide superior results to heavier 3D representations.

These observations indicate adding a modality doesn't guarantee improvement per se. The effect depends on how geometry is represented and connected to the action policy. DP3 uses a structure informative enough to describe space but prevents its processing from dominating total architecture cost.

However, both PerAct and DP3 are trained from scratch as specific policies and don't

directly leverage knowledge inside large pre-trained vision-language models. This difference matters because a significant part of VLA capability comes precisely from semantic representations acquired via massive image and text volumes. The next advance is, therefore, incorporating 3D geometry inside an already pre-trained model, avoiding both the cost of repeating the entire process and the loss of previously acquired knowledge.

PointVLA addresses this by integrating point clouds into VLA models already pre-trained primarily on 2D observations [14]. The proposal stems from a major difference between both data sources: image and RGB trajectory datasets available for pre-training are considerably larger than robotic datasets containing 3D info.

Fully retraining a foundational model using a new modality would mean repeating substantial computational cost and could alter previously learned representations. Conversely, replacing 2D data with smaller-scale 3D datasets would entail giving up visual and semantic info acquired during pre-training.

PointVLA adopts a modular strategy. The vision-language backbone and most of the action expert keep already learned representations, while a new module processes the point cloud and adds its features to certain expert blocks. The goal isn't replacing RGB perception but complementing it with explicit geometric info.

Figure 2.20 shows the general architecture. Images and instruction continue processing through the original VLM. Concurrently, the point cloud transforms into a representation compatible with the action expert's internal space. This representation gets incorporated via additional modules into a selected part of the network.

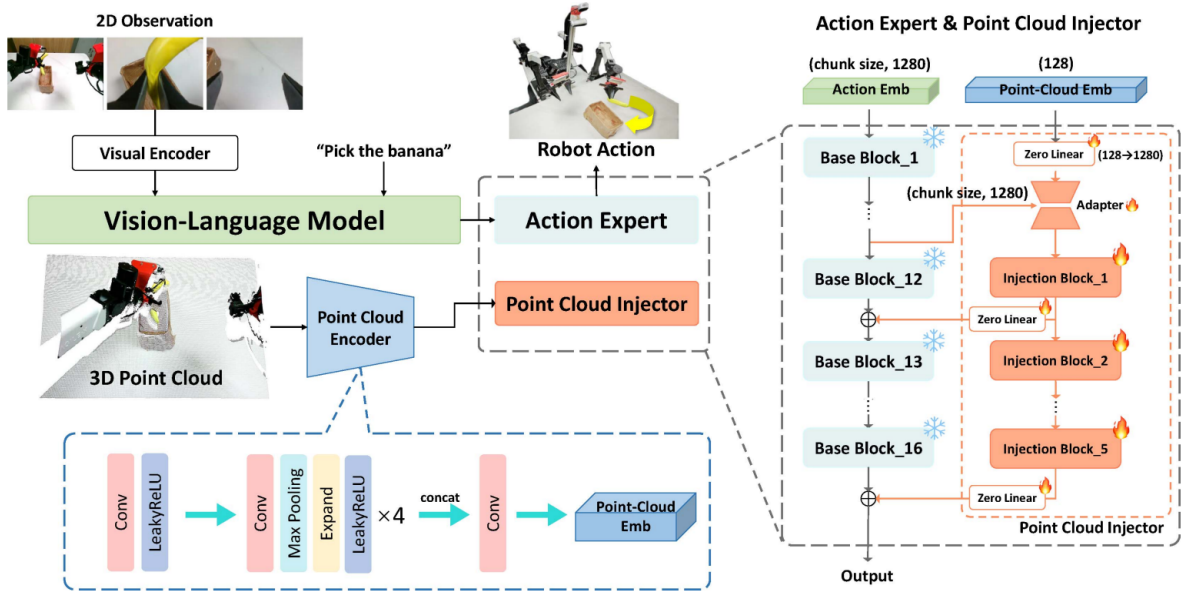


Figure 2.20: General PointVLA architecture. The model preserves the pre-trained VLA's vision-language processing and incorporates a point cloud representation via a 3D module connected to certain action expert blocks [14].

Selecting these blocks constitutes a central part of the proposal. Injecting 3D info into all layers would increase compute cost and broadly alter the expert's internal representations. This strategy could cause interference with capabilities previously learned from 2D data.

2.6. Three-Dimensional Geometric Representation

To reduce this risk, PointVLA uses skip-block analysis. The procedure evaluates action expert block importance by dropping them during inference and observing the performance effect. Blocks whose absence produces minor degradation are deemed more suitable candidates for receiving the new modality.

As depicted in Figure 2.21, the analysis differentiates the expert’s most critical parts from those showing greater adaptability. Injection focuses on the latter, while the rest of the expert is preserved.

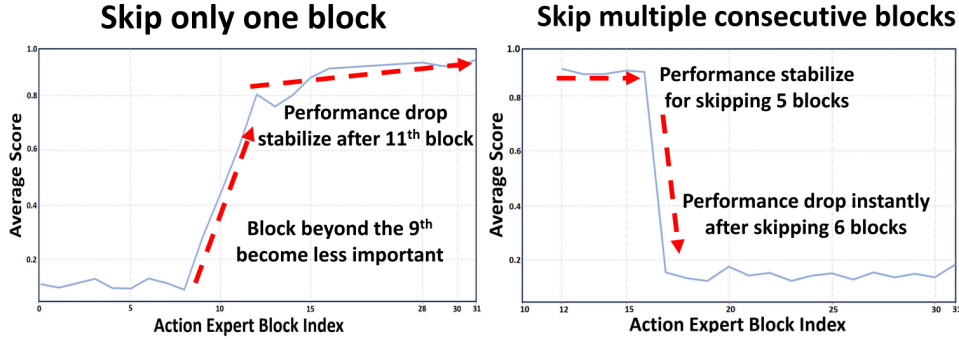


Figure 2.21: Skip-block analysis used in PointVLA. Action expert layer importance is evaluated via omission during inference. Less critical blocks are chosen to receive 3D info, reducing interference on pre-trained representations [14].

Once blocks are selected, point cloud features adapt to their internal dimension and incorporate via an additive operation. This formulation keeps 3D feature extraction separate from the 2D visual encoder and limits the number of components requiring training on new data.

The proposal turns 3D incorporation into a representation compatibility problem. It’s not enough to introduce depth info anywhere in the architecture. The new modality must be added so it contributes to action generation without degrading previously acquired visual and semantic knowledge.

Experiments in the work compare PointVLA against 2D and 3D policies in simulated and real scenarios [14]. Results show improvements over different imitation learning methods, including OpenVLA, Diffusion Policy, and DexVLA, across evaluated setups.

Beyond aggregate performance, evaluations study situations where explicit geometry provides a distinct advantage. One consists of distinguishing a physical object from a photograph of it. From a specific RGB view, both might appear similar, but the point cloud allows identifying that the photograph doesn’t occupy expected volume in space.

Adaptation to work surface height changes is also analyzed. Models based solely on 2D observations might associate learned appearance with a specific spatial layout and output inadequate trajectories when height changes. PointVLA uses observed geometry to adjust the action to the new arrangement.

The work also includes few-shot learning scenarios and longer-duration tasks [14]. Together, these evaluations suggest some 2D VLA limitations aren’t solved merely by increasing data volume, since they stem from lacking a sufficiently explicit geometric description.

The progression from PerAct to PointVLA identifies three stages. PerAct shows a structured 3D representation can aid learning 6-DoF actions. DP3 proves point clouds can also condition continuous generative policies efficiently. PointVLA finally transfers this info into a pre-trained VLA via a modular architecture seeking to preserve prior capabilities.

This last stage is particularly relevant to the present work. Instead of developing a 3D policy from scratch, PointVLA posits geometry can be incorporated as a complementary source within an existing VLA base. This strategy allows simultaneously leveraging semantic knowledge learned via vision and language alongside the spatial info necessary to interact more accurately in the physical environment.

2.7. Synthesis and Work Positioning

The evolution reviewed throughout this chapter shows the convergence of different research lines. Imitation learning, human demonstrations, Transformer architectures, action sequence generation, and spatial representations provide the foundations upon which VLA models integrate vision, language, and robotic control. From this initial formulation, the paradigm evolves towards more general, open, and efficient policies capable of using different action generation mechanisms and new perceptual info sources.

Table 2.1 summarizes the main works analyzed, based on their inputs, the procedure used to generate actions, and their primary contribution. The comparison doesn't aim to establish a performance ranking, as each proposal is evaluated on different platforms, tasks, and datasets.

The comparison identifies two main trends. The first corresponds to the transition from discretized, token-generated actions to continuous blocks obtained via diffusion or flow matching. The second involves expanding info available to the policy via tactile signals, temporal memory, and 3D representations. These lines show VLA evolution depends not just on model size, but also on how motion, temporal context, and environment geometry are represented.

The present work is situated at the intersection between memory and 3D perception. Short-term visual memory and depth info are incorporated into the SmolVLA architecture, both independently and jointly. This strategy maintains the base model's visual and linguistic capabilities and introduces specialized components to better represent occlusions and scene spatial structure, following the modular orientation observed in works like MEM [12] and PointVLA [14].

Cuadro 2.1: Synthesis of the main works reviewed in the state of the art.

Model	Year	Main Input	Action Generation	Main Contribution
PerAct [1]	2022	Language and voxelized RGB-D observations.	Discretized 6-DoF action selection via next best voxel action.	Introduces a structured, language-conditioned 3D representation for Transformer-based manipulation.
ALOHA/ACT [2]	2023	Images, proprioceptive state, and bimanual human demonstrations.	Action blocks generated via a conditioned Transformer and combined with temporal ensembling.	Proposes an affordable teleoperation platform and develops action chunking to reduce error accumulation.
RT-2 [3]	2023	Image and linguistic instruction.	Autoregressive generation of actions discretized as tokens.	Formalizes the VLA paradigm and transfers knowledge acquired via vision-language pre-training to robotic control.
DP3 [13]	2024	Point cloud and proprioceptive state.	Continuous action sequences via a diffusion policy.	Demonstrates the efficiency of a compact 3D representation for generative manipulation policies.
Octo [4]	2024	Images, proprioception, and linguistic instructions or goal images.	Continuous action blocks via a diffusion head.	Develops an open generalist policy adaptable to new observations, action spaces, and platforms.
OpenVLA [6]	2024	Image, linguistic instruction, and robot state.	Autoregressive generation of discretized action tokens.	Provides an open, reproducible VLA compatible with efficient fine-tuning and quantization procedures.
TinyVLA [9]	2024	Images, language, and robot state.	Continuous action sequences via a diffusion policy.	Reduces VLA size and latency and improves data efficiency without relying on large-scale robotic pre-training.
Π_0 [5]	2024	Images, linguistic instruction, and proprioception.	Continuous blocks generated by an action expert trained with flow matching.	Separates vision-language processing from continuous action generation via a specialized module.
PointVLA [14]	2025	Image, language, robot state, and point cloud.	Continuous actions via an expert receiving injected 3D features.	Integrates 3D geometry into a pre-trained VLA via lightweight modules and skip-block analysis.
GR00T N1 [7]	2025	Images, language, robotic state, and heterogeneous data from various sources.	Continuous actions via a Diffusion Transformer with flow matching.	Extends VLA models to multiple morphologies, especially humanoid platforms.
$\Pi_{0.5}$ [8]	2025	Images, language, semantic subtasks, and heterogeneous robotic and web data.	Pre-training with discretized actions and post-training of a continuous expert with flow matching.	Improves generalization in unobserved domestic environments through co-training and hierarchical inference.
VTLA [11]	2025	Vision, tactile sequences, and linguistic instruction.	Tokenized actions later refined via DPO.	Incorporates tactile perception into a VLA to solve contact-intensive insertion tasks.
SmoVLA [10]	2025	Multiple RGB images, linguistic instruction, and proprioceptive state.	Continuous blocks via an action expert trained with flow matching.	Combines a compact architecture, community data, affordable platforms, and asynchronous inference.
MEM [12]	2026	Current observation, recent visual memory, text memory, and instruction.	Continuous actions conditioned by multi-scale memory and semantic subtasks.	Introduces short-term visual memory and long-term linguistic memory for prolonged and partially observable tasks.

Capítulo 3

Reference Architecture and System Fundamentals

This chapter presents the theoretical elements necessary to understand the operation of the SmolVLA model [10] and to justify the proposals developed in this work. First, imitation learning is addressed, the principle upon which the collection of robotic demonstrations is based. Next, the flow matching technique used to train the action expert is described. Subsequently, the general architecture of SmolVLA is detailed, analyzing the integration between the vision-language module and continuous action generation. Finally, the principles of depth perception and short-term visual memory are presented, key elements for the formulation of the variants described in Chapter 4.

3.1. Imitation Learning

Imitation learning allows training a control policy from demonstrations provided by a human operator, or even by an already trained policy. These demonstrations are usually obtained via teleoperation. The operator controls the robot while camera images, the system’s internal state, and the executed actions are synchronously recorded. An instruction describing the performed task is also associated with each demonstration.

Each demonstration constitutes an episode formed by a temporal sequence of observations and actions. Images describe the environment and object positions, while the proprioceptive state collects the robot’s internal information, such as its joint positions or gripper aperture. These data constitute the policy’s training set, with actions as the objective function and the rest of the data as the context to condition this function.

During deployment, the robot operates in a closed loop. The policy receives an observation, generates an action block, executes a part of it, and observes the scene again before continuing. This update allows adapting the movement to changes produced during execution.

However, an important difference arises here between the training and inference phases. During data collection, the policy only observes correct movements performed by

the operator. Conversely, when the robot operates on its own, a small error when executing an action slightly changes the scene. If the model has not seen that situation before, this small failure accumulates and can cause the robot to completely deviate from the task.

For this reason, the quality and above all the variety of the demonstrations are fundamental. It is not enough to record the same repeated trajectory; it is advisable to include different object positions and examples where the operator themselves corrects a failed grasp or a deviation. Thus, the robot learns to react and recover when execution does not go perfectly as in theory [2], [10].

3.2. Flow Matching

Flow matching is a generative technique that allows progressively transforming a simple data distribution (such as Gaussian noise) into a target distribution to be replicated [15].

To understand the idea intuitively, flow matching works by defining a vector field that changes over time. Figure 3.1 shows a simplified diagram of this process: the initial samples (the blue dots representing noise) move and adopt the shape of the target distribution (the red figure) as time advances, guided by the vectors at each point. At each intermediate stage, the model calculates where it must move the samples to bring them closer to the desired final shape.

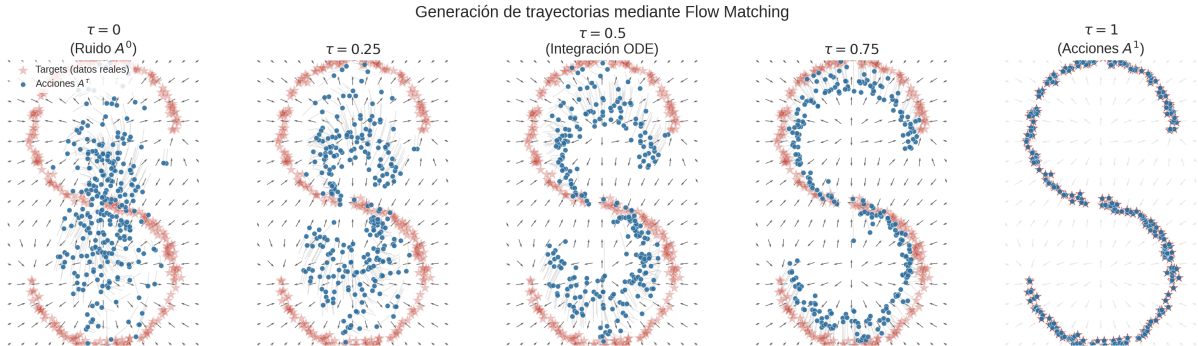


Figura 3.1: Conceptual evolution of the flow matching process. Own elaboration based on [15].

However, in the field of VLAs, an adaptation of this concept is used. In a robot, the goal is not to always generate the same target distribution, but a specific response according to what is happening at any given moment. Therefore, the action expert applies conditional flow matching, which is represented as:

$$v_{\psi} = f_{\psi}(x_{\tau}, c, \tau), \quad (3.1)$$

where x_{τ} is the intermediate state of the action block, τ indicates the stage or progress of the transformation process (the “time” in the initial explanation), c is the context vector containing the visual information and the linguistic instruction, and v_{ψ} is the vector field predicted by the model f_{ψ} with which we would obtain the next x_{τ} .

In robotic control systems, the target distribution is formed by the action sequences of real demonstrations (whose collection is better explained in Section 3.1). During training, the model receives the context and combines the real actions with different levels of noise. In this way, it learns to predict the necessary corrections to transform that noise into movements consistent with the task.

SmolVLA applies this principle directly in its action expert [10]. The vision-language model processes the images and the instruction to build the context c , while the expert receives this information along with the robot’s proprioceptive state. During inference, the process begins with a completely random action block that the action expert iteratively corrects following the predicted vectors. Upon completing these refinement steps, the initial noise is transformed into a smooth and continuous sequence of movements, ready to be executed by the robot.

3.3. SmolVLA Architecture

SmolVLA is a VLA architecture that combines a compact vision-language model with an expert specialized in continuous action generation [10], as illustrated in Figure 3.2. To do this, a SigLIP-based visual encoder processes the camera images, while the SmolLM2 language model receives and synthesizes these visual features along with the textual instruction and the robot’s proprioceptive state. At each inference step, this block builds a unified multimodal context in a single pass at the beginning of the cycle, which allows optimizing computation time and decoupling dense scene perception from action generation.

For its part, the action expert is trained via flow matching and operates in parallel on the obtained representations. Instead of predicting a single action, it generates a continuous sequence of future actions by applying action chunking (a concept introduced in Figure 2.3 of Chapter 2). Its initial input is a Gaussian noise sample that is progressively refined over ten iterations. To guide this transformation, the action expert interleaves self-attention and cross-attention layers. Both stages employ the same attention operator, differing only in the source of the input variables.

In self-attention, the queries, keys, and values are extracted from the latent representation of the action block itself, denoted as H_A , allowing the different elements of the trajectory to be related to each other:

$$H_{\text{self}} = \text{Attention}(Q = H_A, K = H_A, V = H_A). \quad (3.2)$$

In cross-attention, the same function is used but varying the source of the keys and values. The queries (Q) are derived from H_A , while the keys (K) and values (V) come from the multimodal context generated by the VLM, denoted as H_{VLM} :

$$H_{\text{cross}} = \text{Attention}(Q = H_A, K = H_{\text{VLM}}, V = H_{\text{VLM}}). \quad (3.3)$$

In this way, self-attention internally coordinates the flow of the action block, while cross-attention acts as the conditioning mechanism of the flow matching process, allowing said actions to extract the visual, semantic, and proprioceptive information necessary to guide action generation.

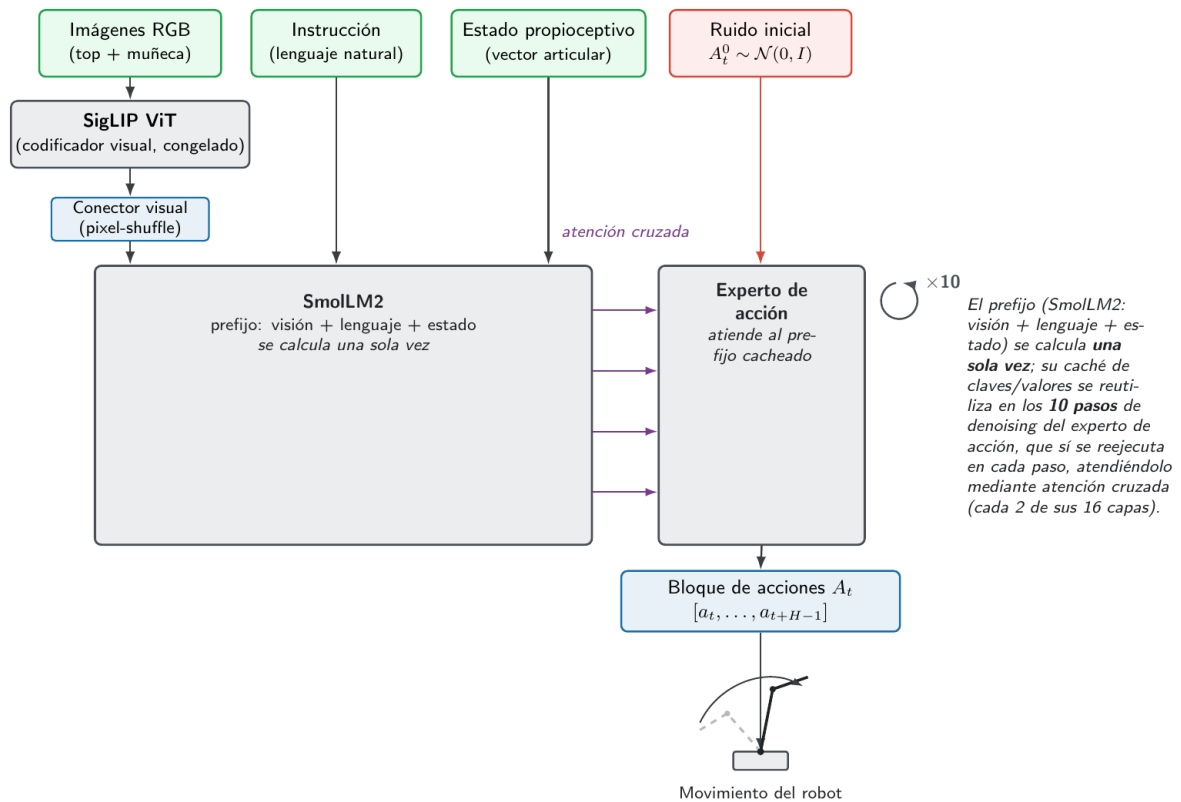


Figura 3.2: Simplified architecture of SmolVLA. The vision-language model processes RGB images, the instruction, and proprioception to generate a multimodal context that constantly conditions the action expert in action generation. Own elaboration based on [10].

3.4. Three-Dimensional Representation and Feature Injection

To estimate the scene’s depth in this work, a stereo camera is used, which captures the scene from two slightly separated viewpoints. After aligning both images, the position of the same point in the left and right views is sought. The horizontal difference between both positions is called disparity.

The idea is similar to observing an object alternating between one eye and the other: nearby elements appear to shift more than distant ones. Therefore, a large disparity indicates that the point is close to the camera, while a small disparity indicates that it is further away.

Knowing this disparity and the camera’s calibration parameters, the distance to each point is calculated. The result is transformed into a three-dimensional point cloud, where each point represents a position in the scene. This cloud is filtered to remove incorrect values and is downsampled before being processed by an encoder.

The encoder converts the cloud into compact features that represent the environment’s geometry. These features complement those obtained through RGB images: while the image provides visual and semantic information, the point cloud accurately describes the depth and spatial arrangement of objects, avoiding trajectory errors during manipulation [14].

To incorporate this information into the model, the three-dimensional features are adapted to the internal dimension of the action expert and are added to its representations via a residual sum:

$$\tilde{h}^{(l)} = h^{(l)} + \alpha_l r_{3D}^{(l)}, \quad (3.4)$$

where $h^{(l)}$ represents the block’s original features, $r_{3D}^{(l)}$ contains the adapted geometric information, and α_l controls its contribution.

In this way, three-dimensional information is introduced as an additional signal without replacing the original visual and linguistic processing. This principle, inspired by PointVLA, allows adding depth to SmolVLA and studying whether an explicit representation of geometry improves action generation [14].

3.5. Short-Term Visual Memory

An exclusively reactive control policy generates its actions based solely on the current instant’s observation. However, this approach proves insufficient when the environment is partially observable, as occurs when an object is temporarily hidden by another scene element or by the robot’s own gripper. In these situations, a single image does not contain enough information to determine the task state, since two visually identical observations can correspond to different moments in the execution and require completely different actions.

To solve this problem, short-term visual memory expands the model’s input by storing a sequence with the most recent observations. Mathematically, this memory window $O_t^{K,\Delta}$ at the current time t is defined as:

$$O_t^{K,\Delta} = [o_{t-(K-1)\Delta}, \dots, o_{t-\Delta}, o_t], \quad (3.5)$$

where o_t is the current instant's image, K is the total number of frames we keep in memory, and Δ is the temporal stride or separation between them (for example, taking an image every certain number of steps or every certain milliseconds).

By maintaining this small sequence of past images, the robot can remember where an object it just covered with its own hand was, check if a grasp was successful, or interpret where an element was moving. For the model to leverage these observations, the representations of the different frames are connected through a temporal attention mechanism, allowing the network to learn to pay attention only to past moments that are truly relevant to deciding the current movement.

Therefore, the function of this short-term memory is not to store the complete history of a prolonged task, but to preserve visual and motion details during a short interval of a few seconds. This capability is especially useful for overcoming temporary visibility losses and self-occlusion problems [12].

In this way, the temporal dimension is modularly incorporated onto the environment's representation, respecting the original vision and language processing. This integration allows extending the SmolVLA base model to study whether retaining recent features favors learning more robust control policies [12].

Capítulo 4

Methodology

This chapter details the methodology developed in this work. First, the experimental environment, both physical and digital, is described. Next, the data collection process is explained, and the four SmolVLA variants developed are presented along with how they were trained. Finally, both the ablation test on the action expert layers and the final evaluation are described. The results of these evaluations are presented and analyzed in Chapter 5.

4.1. Experimental Environment

For this work, the same physical platform has been used for both data collection and final evaluations. This platform uses the SO-101 robotic arm¹, developed by The Robot Studio in collaboration with Hugging Face. It is a 6-degree-of-freedom robot fully integrated into the LeRobot ecosystem, which facilitates the deployment of SmolVLA as it has native support and a base model already pre-trained on this hardware. Figure 4.1 shows what this robot looks like alongside its teleoperation leader arm.

Regarding the sensors, there are three cameras which can be seen in Figure 4.2. The setup consists of two active cameras per experiment: a fixed top camera that observes the entire scene, and a camera mounted on the robot's wrist. The wrist alternates between a monocular RGB camera, used for the ablation test explained in Subsection 4.4.1, and a stereo camera for the final evaluation, described in Subsection 4.4.2.

For the task, a scenario delimited by the dimensions of an A3 paper is proposed. In it, there are four figures that the robot must manipulate, a bin to deposit the stars, and an area marked on the paper to deposit the cubes. The workspace is divided into nine delimited positions, as shown in Figure 4.3, numbering the columns from 1 to 3 and the rows with the letters A, B, and C. The instruction passed to the model does not change at any point and is stated as:

Put the stars in the bin and put the cubes on the marked area.

This configuration seeks, on the one hand, to structure the workspace to standardize the initial positions of the objects in the scene during evaluations and, on the other,

¹<https://github.com/TheRobotStudio/SO-ARM100>



Figura 4.1: SO-101 arms for teleoperation: follower with gripper (left) and leader (right). Images from <https://huggingface.co/docs/lerobot/so101>.



Figura 4.2: System cameras: fixed top RGB (left), monocular wrist RGB (center), and SVPRO stereo with its calibration and side-by-side capture (right).



Figura 4.3: Workspace discretization: 3×3 grid (positions A1-C3) for object origin and fixed destination area.

to promote a higher probability of self-occlusion by deliberately placing the robotic arm close to the manipulation zone.

For the recording of demonstrations and evaluations, the LeRobot library itself is used. The storage of datasets and checkpoints is managed through Hugging Face, while training monitoring, along with the logging of metrics and losses, is performed in Weights & Biases.

4.2. Data Collection and Preparation

Two main datasets have been created for this work: one used in the ablation test and another in the final evaluation. Both consist of a total of 120 episodes with different configurations each, varying the position, rotation, and proximity of the figures to provide sufficient variability to the dataset. For data collection, the experimental platform was used by manually controlling the movements through the leader arm.

The main difference between both datasets lies in the wrist camera used and the total recording time: the ablation dataset uses the monocular RGB camera (accumulating about 103 minutes), while the final dataset uses the monochrome stereo camera (about 87 minutes). From this second set, a variant with depth was generated, whose preprocessing is shown in Figure 4.4.

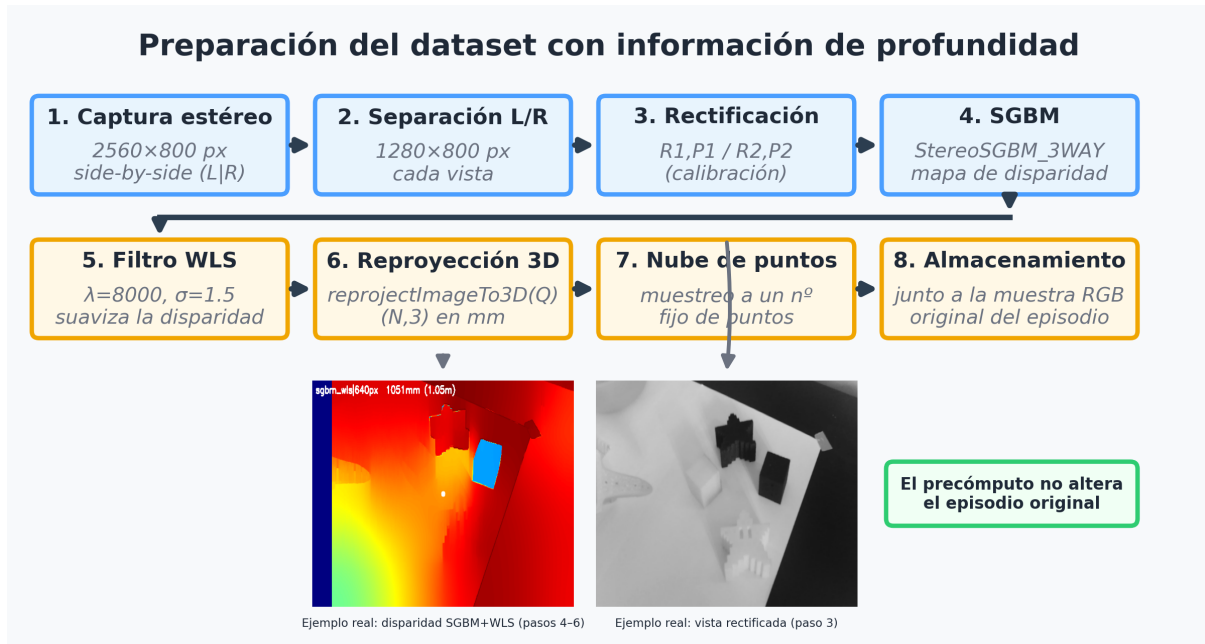


Figura 4.4: Processing pipeline for the depth dataset: stereo view rectification, disparity map calculation, and point cloud reprojection.

4.3. SmolVLA Variants

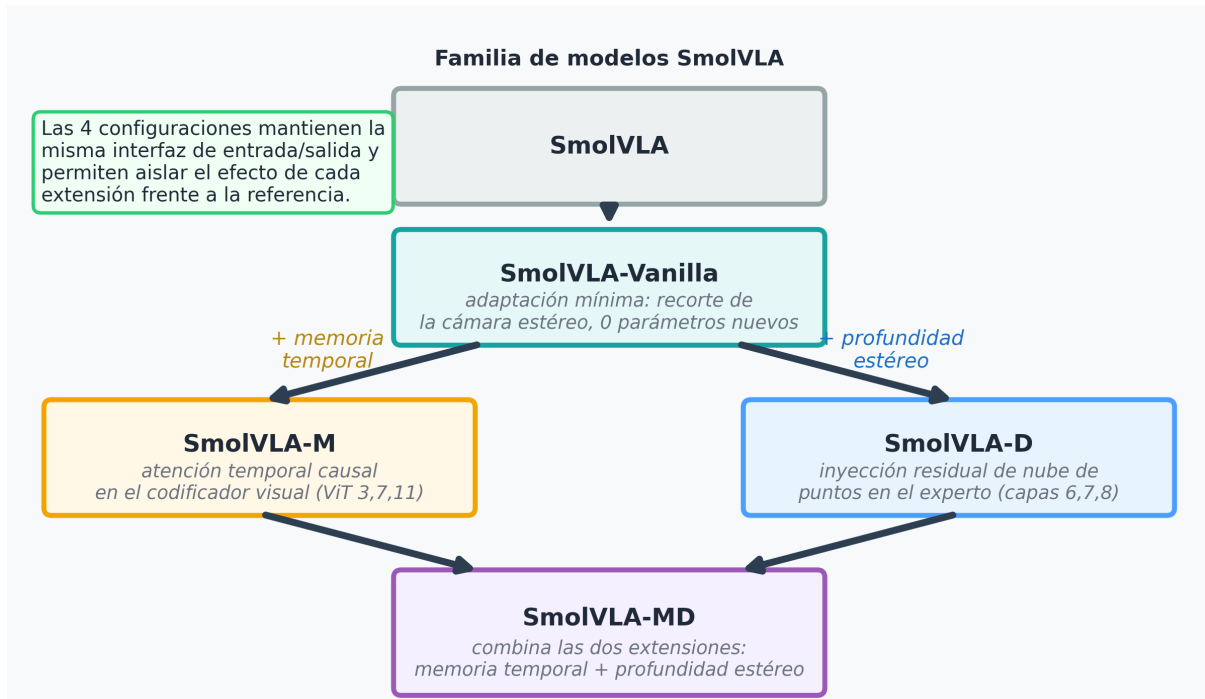


Figura 4.5: Relationship and hierarchy of the SmolVLA model family developed from the base checkpoint.

All architectures developed in this work are based on the SmolVLA base model available in the LeRobot library. For the ablation study on the action expert layers (Sub-

section 4.4.1), this original model is used without modifying the network, solely evaluating the progressive removal of its layers.

In contrast, for the final evaluation, four variations are designed that introduce specific structural modifications to address memory and spatial perception problems. Figure 4.5 summarizes the hierarchical relationship between the base model and the four developed configurations (SmolVLA-Vanilla, SmolVLA-M, SmolVLA-D, and SmolVLA-MD).

4.3.1. SmolVLA-Vanilla

This variant is conceived as the reference architecture for the final evaluation. It keeps the original structure of SmolVLA intact without adding additional parameters, guaranteeing that the measured performance corresponds exclusively to the model’s base capacity.

The only modification lies in a preprocessing module designed to manage the visual input from the wrist camera. Since this stereo sensor delivers a continuous capture with the left and right views concatenated, the module automatically crops the image and selects only the right half to feed into the visual encoder. In this way, the same physical hardware is maintained in all experiments without altering the monocular input format required by the original network.

As observed in Figure 4.6, this minimal adaptation constitutes a common modification to all four developed variants. To simplify the diagrams of the subsequent models, this cropping box is not explicitly shown in the rest of the figures, although it remains active in all experiments to guarantee homogeneous and mutually comparable conditions.

4.3.2. SmolVLA-M

SmolVLA-M incorporates short-term visual memory through an adaptation of the video encoder proposed in MEM [12]. As detailed in Section 3.5, instead of processing only the current frame, the visual encoder receives a sequence of $K = 3$ images with a temporal stride s of approximately 2.5 seconds. This setup forms a total temporal window of 5 seconds that preserves MEM’s coverage, albeit with a lower temporal resolution due to VRAM restrictions during training.

The visual representations are combined via causal temporal attention in layers 3, 7, and 11 of the visual encoder. This distribution replicates the MEM pattern by interleaving a temporal operation every four ViT blocks, allowing the history to be integrated at different abstraction levels without transforming all layers into spatio-temporal operations.

The output of each modified block is calculated using the following residual connection:

$$\tilde{H}^{(l)} = H_{\text{esp}}^{(l)} + \alpha_l H_{\text{temp}}^{(l)}, \quad l \in \{3, 7, 11\}, \quad (4.1)$$

where $H_{\text{esp}}^{(l)}$ represents the original spatial activation and $H_{\text{temp}}^{(l)}$ the information extracted from past observations. The conceptual scheme of this architecture is shown in Figure 4.7.

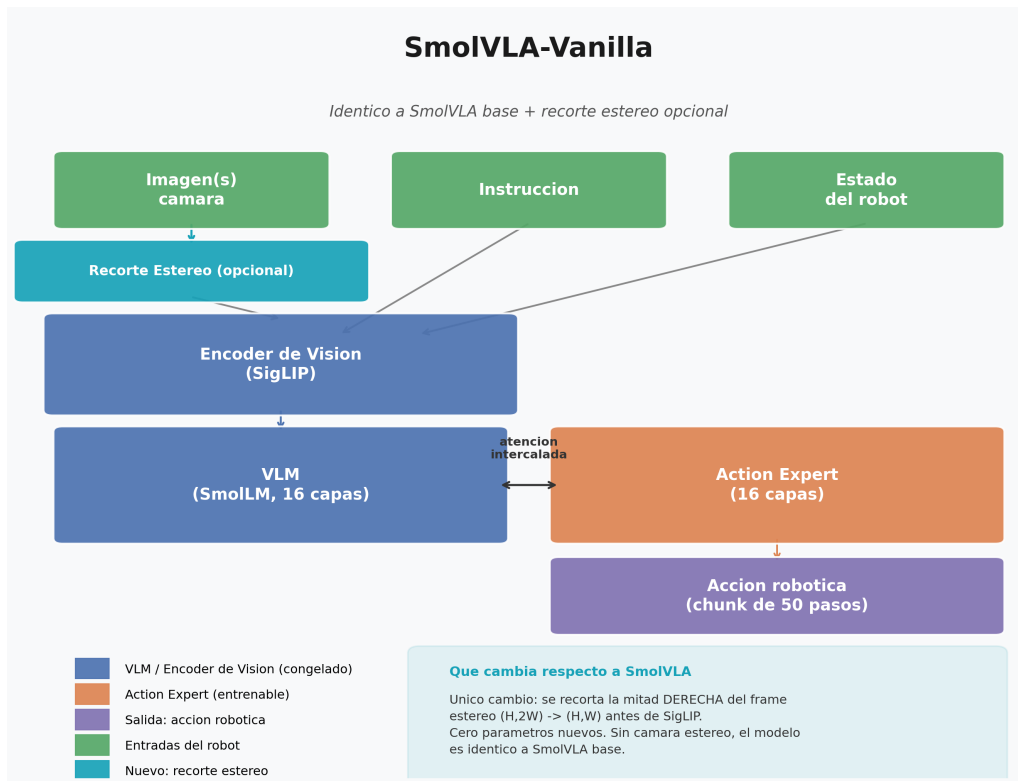


Figura 4.6: General overview of the SmolVLA-Vanilla architecture.

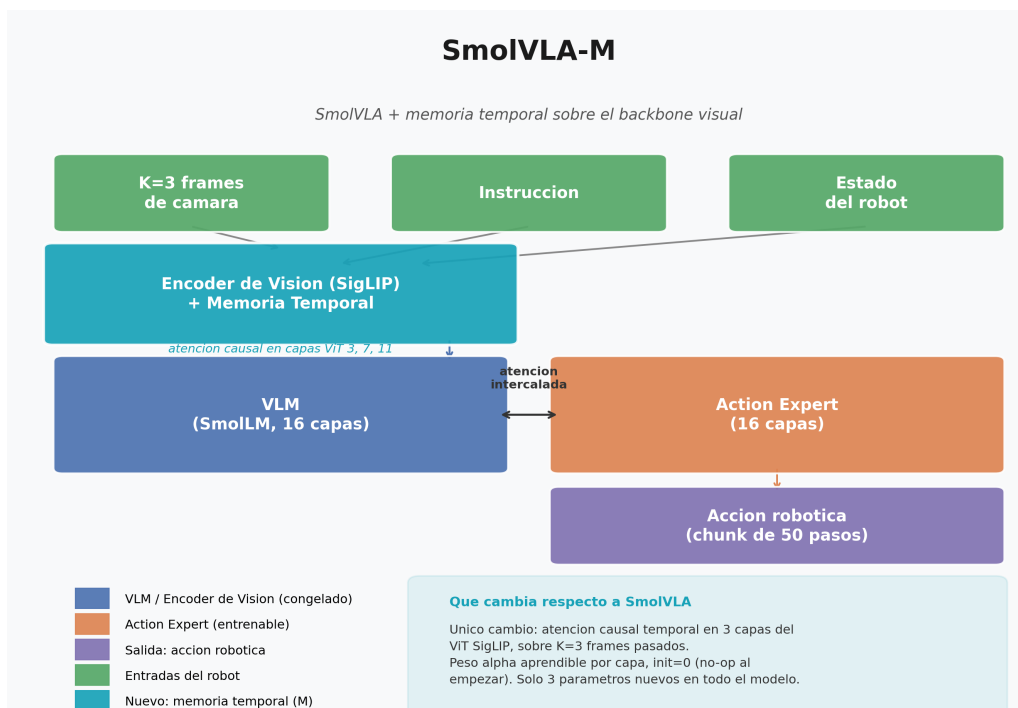


Figura 4.7: SmolVLA-M architecture: integration of causal temporal attention in layers 3, 7, and 11 of the visual encoder.

4.3.3. SmolVLA-D

As detailed in Section 3.4, 3D point clouds are obtained from the stereo camera capture. To inject this spatial information into the action expert of SmolVLA-D, we follow the PointVLA strategy [14] by dividing the process into two modules.

First, a PointCloudEncoder processes the three-dimensional coordinates to extract a compact representation that summarizes the scene's geometry. Then, the DepthAdapter is responsible for projecting this representation to adjust its dimension to that of the action expert. Finally, the resulting signal is injected via a residual sum only into blocks 6, 7, and 8 of the network.

The choice of layers 6, 7, and 8 is decided after performing the ablation study, thus identifying which blocks are most susceptible to incorporating three-dimensional information. Figure 4.8 shows the general architecture of SmolVLA-D, the stereo branch, and the exact injection point within the action expert.

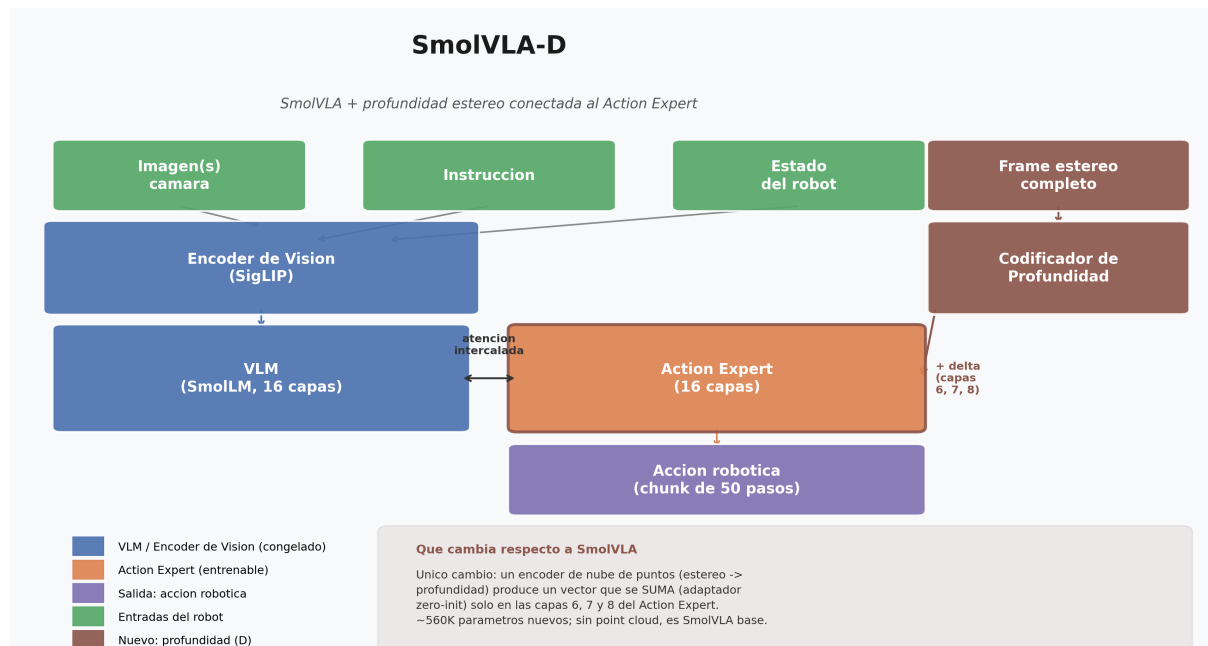


Figura 4.8: SmolVLA-D architecture.

4.3.4. SmolVLA-MD

SmolVLA-MD simultaneously integrates the improvements developed in SmolVLA-M and SmolVLA-D. Since temporal memory acts in the visual encoder (blocks 3, 7, and 11) and three-dimensional information is injected into the action expert (blocks 6, 7, and 8), both extensions are completely independent and do not overlap.

With this variant, we seek to verify if combining temporal memory and 3D information improves the robot's dexterity compared to using them separately. Figure 4.9 shows how both paths are integrated into the model.

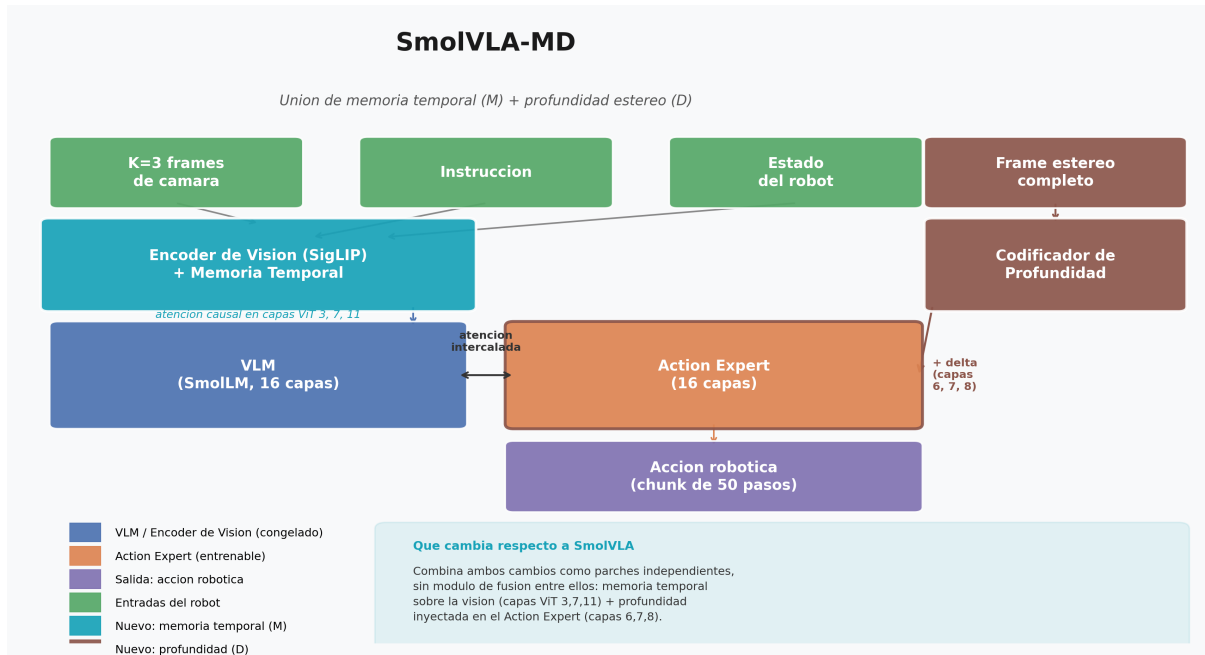


Figura 4.9: SmoIVLA-MD architecture.

4.3.5. Trainings

The models are trained starting from the SmoIVLA base weights available in LeRobot, initializing only the new extensions to zero. During the process, the ViT and language model are frozen, so that only the action expert and the additional modules (the point cloud encoder and 3D adapter in SmoIVLA-D, and the three temporal attention scalars in SmoIVLA-M) are optimized. The original objective function based on flow matching is maintained without auxiliary losses, meaning all learning comes from action prediction.

For optimization, AdamW is used with a maximum learning rate of 10^{-4} (linear warmup of 1 000 steps and cosine decay to 2.5×10^{-6}), a weight decay of 10^{-10} , and gradient clipping at 10. Furthermore, the data augmentation option on the images is disabled so as not to alter the stereo camera cropping.

The four models were trained sequentially on an NVIDIA H100 GPU (80 GB), setting a target budget of 1,440,000 samples and seed 42. To avoid exclusive dependence on geometry in the depth variants (D and MD), a 20 % random dropout of the depth data is applied. Table 4.1 summarizes the configuration and final status of the runs.

Cuadro 4.1: Summary of training configuration and execution for the final models (NVIDIA H100 80GB).

Model	Batch	Steps (Actual / Planned)	Epochs	Time	Checkpoint	Status
SmoIVLA-Vanilla	64	22 500/22 500	11.01	4 h 47 min	Step 22 500	Complete
SmoIVLA-D	64	22 500/22 500	11.01	5 h 21 min	Step 22 500	Complete
SmoIVLA-M	32	24 000/45 000	5.87	8 h 34 min	Step 24 000	Interrupted
SmoIVLA-MD	32	24 000/45 000	5.87	7 h 25 min	Step 24 000	Interrupted

Due to the computational cost of processing three frames per sample in the memory variants (M and MD), their batch size was reduced to 32 and the planned number of steps was increased. As shown in Figure 4.10, SmolVLA-M and SmolVLA-MD were interrupted at step 24 000 due to budget restrictions. Although the loss continued to decline smoothly, the marginal gain did not justify the cost of completing the scheduled cycle, so the last available checkpoint was selected for evaluations.

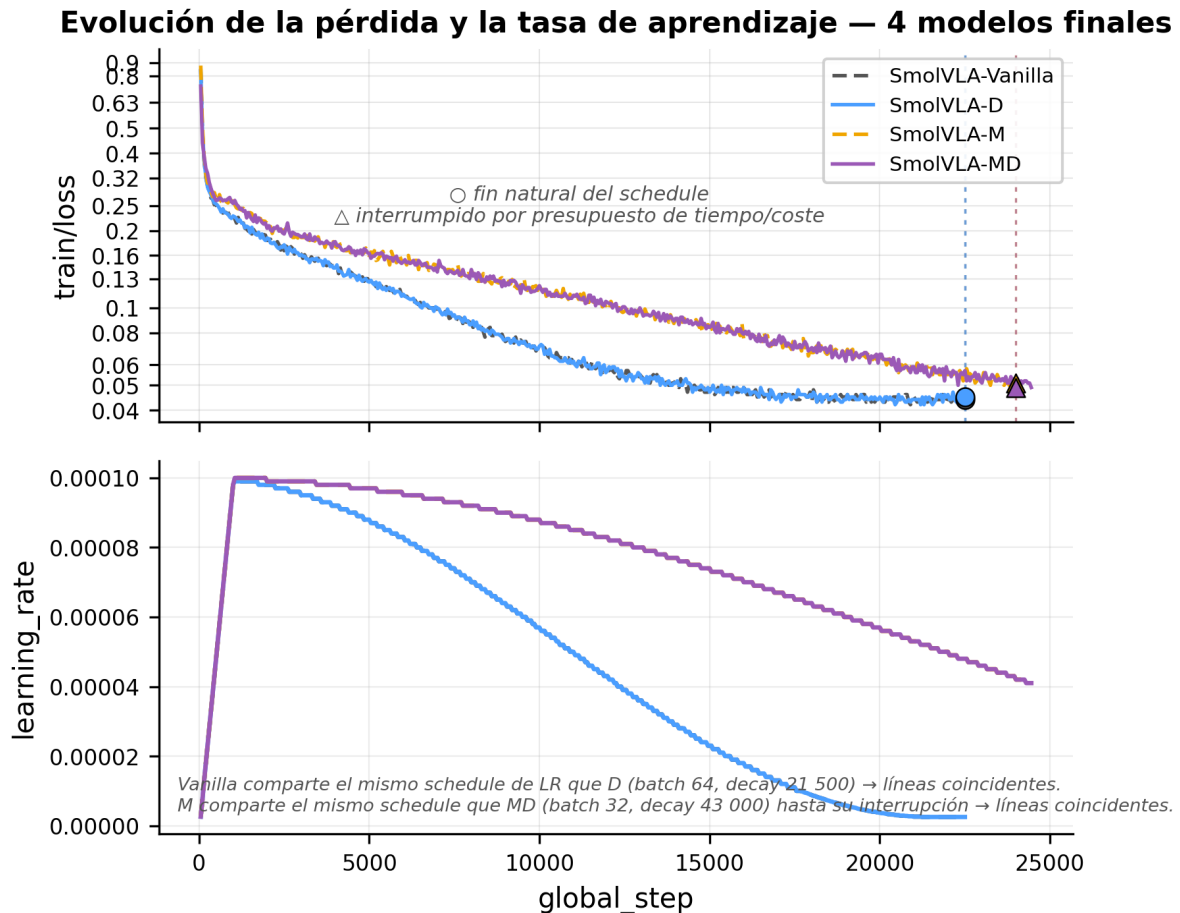


Figura 4.10: Evolution of training loss and learning rate during the training of the four final configurations. The vertical lines indicate the completion or interruption point of each run.

4.4. Experimental Strategy

4.4.1. Layer Ablation Test

As already explained in Subsection 4.3.3, the depth information injection mechanism is inspired by the PointVLA strategy [14]. Similarly, an ablation study on the action expert is proposed before developing the SmolVLA-D and SmolVLA-MD variants. The goal of this experiment is to identify in which blocks of the action expert the injection of new information is least disruptive, ensuring that the three-dimensional signal is integrated without degrading the capabilities previously learned by the model.

The analysis is performed exclusively on the 16 layers of the action expert, keeping the rest of the SmolVLA components frozen and intact. To carry this out, a slight modification was implemented in the model’s original class in LeRobot that allows disabling a specific number of layers. In this context, disabling a layer consists of making it return exactly the input embedding through an identity operation, nullifying its effect on the network without altering tensor dimensions.

Cuadro 4.2: Configuration of the training used as the baseline for the ablation.

Parameter	Value
Base model	lerobot/smolvla_base
Batch / Samples	16 / 1 440 000 (7.71 epochs)
Steps (Total / Warmup / Decay)	90 000 / 1 000 / 86 000
Trainable parameters	Action expert (VLM frozen)
Hardware / Time	NVIDIA RTX 5060 Ti (16 GB) / 14 h 13 min

To evaluate the robot’s behavior fairly in each experiment, a test under identical conditions was designed for all configurations. The test consists of three object layouts with increasing difficulty, evaluating five repetitions per scene to mitigate randomness, and with a time limit of 120 seconds per run.

The ablation test was executed in two phases. In the first, each of the 16 action expert layers was omitted individually to measure its degradation compared to the full model, thus identifying the regions with the highest tolerance to modifications. In the second, after identifying the most stable region, the simultaneous omission of contiguous blocks was evaluated using the configurations {6, 7}, {6, 7, 8}, {7, 8, 9}, and {9, 10, 11}. This second phase is essential for verifying the network’s degree of redundancy and determining the maximum extent of the window over which to inject the depth adapter without destabilizing the control policy.

To evaluate each run, a common rubric from 0 to 9 points was used, assigning 2 points for each of the 4 figures (grasping and placement) and an additional point for completing the test cleanly. From this result, a normalized score between 0 and 1 is obtained for each attempt. Subsequently, graphs are generated compiling the absolute success rate (reaching 9 points), the mean of the 15 runs, and the dispersion per experiment, along with additional diagnostic metrics such as failure modes, manipulation phase, or policy freezing.

For the ablation study, a SmolVLA model was trained from the original base weights using the AdamW optimizer. Table 4.2 summarizes the detailed configuration of this training, and Figure 4.11 shows the evolution of the loss in relation to the learning rate.

4.4.2. Final Evaluation

The objective of the final evaluation is to analyze the impact of the memory and depth modifications described in Section 4.3 compared to the SmolVLA-Vanilla reference architecture, explained in Subsection 4.3.1.

To evaluate temporal memory and 3D discrimination, the experiment introduces two scenario conditions: standard manipulation tests and tests that incorporate a printed

Evolucion de la perdida y la tasa de aprendizaje — modelo de ablacion por capas

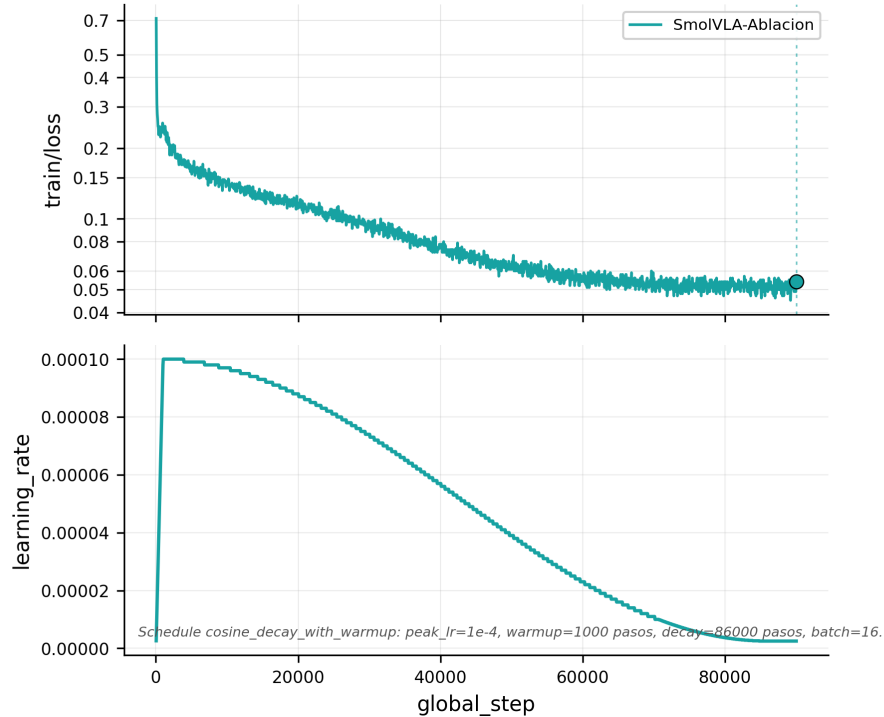


Figura 4.11: Evolution of training loss and learning rate during fine-tuning of the base model for the ablation study.

photograph of one of the objects as a 2D visual distractor. In a design of 12 runs per model on a fixed workspace layout (see Figure 4.3 for reference), real objects are strategically distributed to force the robot's own arm to occlude them during grasps. The first 6 runs evaluate this self-occlusion to measure whether temporal memory allows remembering the position of an object after it becomes hidden. In the remaining 6 runs, the same spatial configuration is maintained, and the printed photograph is added as an OOD element, analyzing whether 3D perception allows discerning the flat, volumeless image from the actual three-dimensional objects.

The evaluation of each run is structured through a rubric that measures the robot's basic dexterity, as well as its memory and 3D perception capabilities. The test initially awards 2 points for each of the 4 figures (grasping and placement) and an extra point for cleanly finishing the task. Based on this, 2 memory points are awarded in all runs (resuming the object after occlusion and completing it) and, in the 6 tests with the photograph, 2 additional 3D perception points (directing the first intention to the real object and avoiding attempting to grasp the printed image). To allow direct comparison between scenarios, the total result is normalized between 0 and 1 by dividing the obtained score by the episode's maximum possible score (11 points without photograph and 13 with it), as synthesized in Table 4.3. Finally, auxiliary diagnostic metrics are recorded, such as failure modes, manipulation phase, or control policy freezing.

A valid run is considered a complete success only when it reaches the scenario's maximum score (11 or 13 points depending on the run), being considered a partial success in any other case where only part of the task is completed (e.g., correct mani-

Cuadro 4.3: Evaluation rubric breakdown according to scenario condition.

Evaluation Category	Without photograph	With photograph
Basic dexterity (4 objects + clean finish)	9 pts	9 pts
Temporal memory (after self-occlusion)	2 pts	2 pts
3D perception (resistance to 2D distractor)	—	2 pts
Maximum score	11 pts	13 pts

pulation of at least one figure). From these runs, graphs and descriptive metrics are generated, such as the mean score, dispersion, or success rate, along with the analysis of failure modes, serving as the basis for the discussion of results in Chapter 5.

Capítulo 5

Results and Discussion

This chapter first presents the results obtained in the action expert layer ablation test (Subsection 4.4.1) along with the design decision derived from them. Next, the results and discussion of the final evaluation on the real robot, whose experimental protocol was described in Subsection 4.4.2, are detailed.

5.1. Layer Ablation Test Results

This section analyzes the sensitivity of the action expert layers to identify the optimal place to inject depth information. The study was divided into two phases: the analysis of individual layers and the ablation of groups of contiguous layers.

In the first phase regarding individual layer sensitivity, Figure 5.1 shows the impact of disabling each layer separately against the original model's performance. Two clearly differentiated behaviors are observed in the results. On the one hand, the layers highlighted in red turn out to be critical, since disabling them drastically degrades the mean score below 0.25 and reduces the success rate to zero. On the other hand, there is a tolerance zone between layers 6 and 11 where the architecture maintains a mean score above 0.40. This continuous band is identified as the candidate region for performing the 3D signal injection.

In the second phase of ablating contiguous sets, Figure 5.2 compares the observed performance against the theoretically expected performance when removing multiple simultaneous layers. As the disablement window shifts towards the end of the network, the penalty on the policy increases. After this trial, the sets {6, 7} and {6, 7, 8} prove to be the most stable in the test, preserving the best average performance levels.

To make the final decision between the candidates {6, 7} and {6, 7, 8}, the error distribution presented in Figure 5.3 was examined. Although the set {6, 7} maintains a residual complete success rate above zero, it generates a high proportion of severe policy freezings. In contrast, the {6, 7, 8} region presents a virtually identical mean score and significantly reduces freezings, exhibiting softer failures such as inaccurate approaches or grasps. Therefore, the {6, 7, 8} region is selected to perform the depth injection, as it offers a wider space for the adapter and dynamic behavior less prone to collapse.

5.1. Layer Ablation Test Results

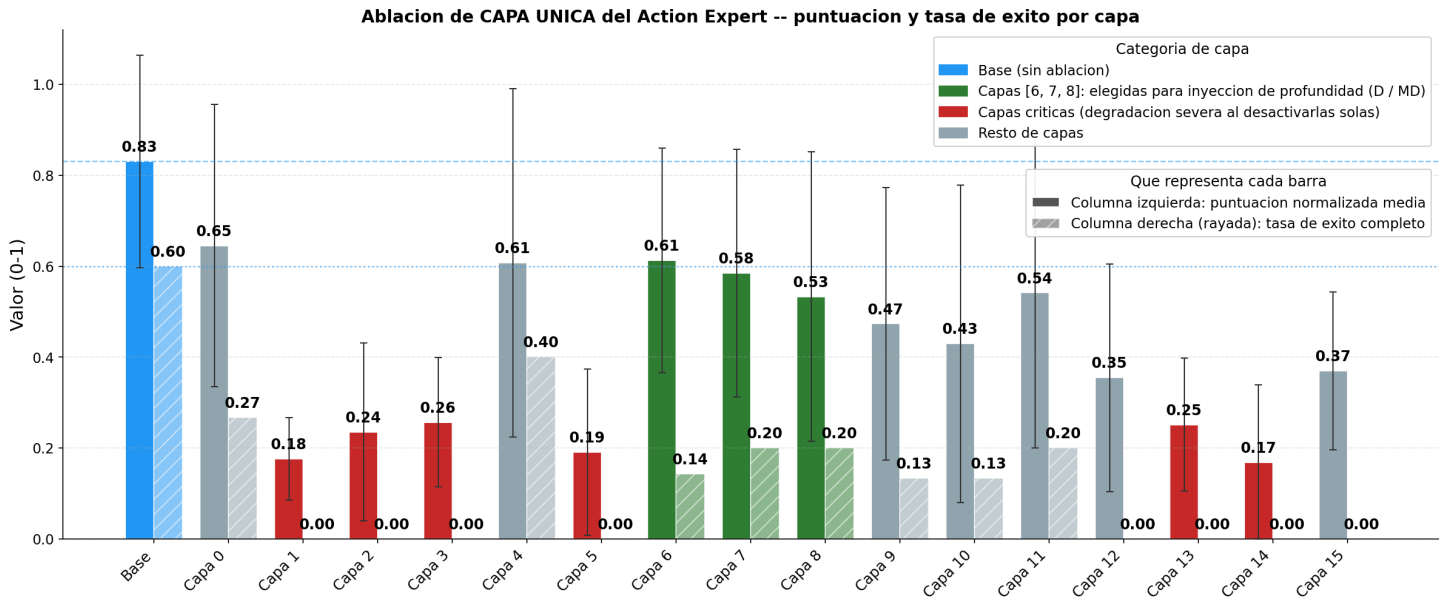


Figura 5.1: Mean normalized score and complete success rate obtained when individually disabling each action expert layer. Error bars represent the standard deviation of the score.

La degradacion combinada es SUPER-ADITIVA (peor de lo esperado) y empeora fuera de 6-8

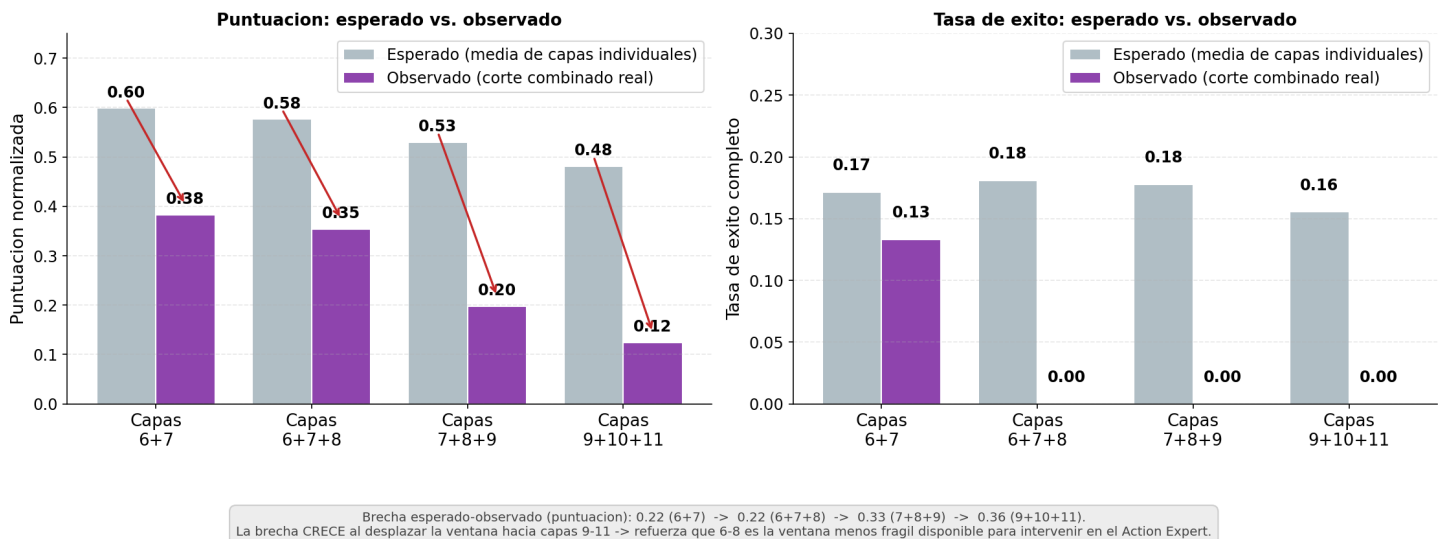


Figura 5.2: Comparison between the expected performance based on individual ablations and the observed performance when disabling several contiguous layers.

Results and Discussion

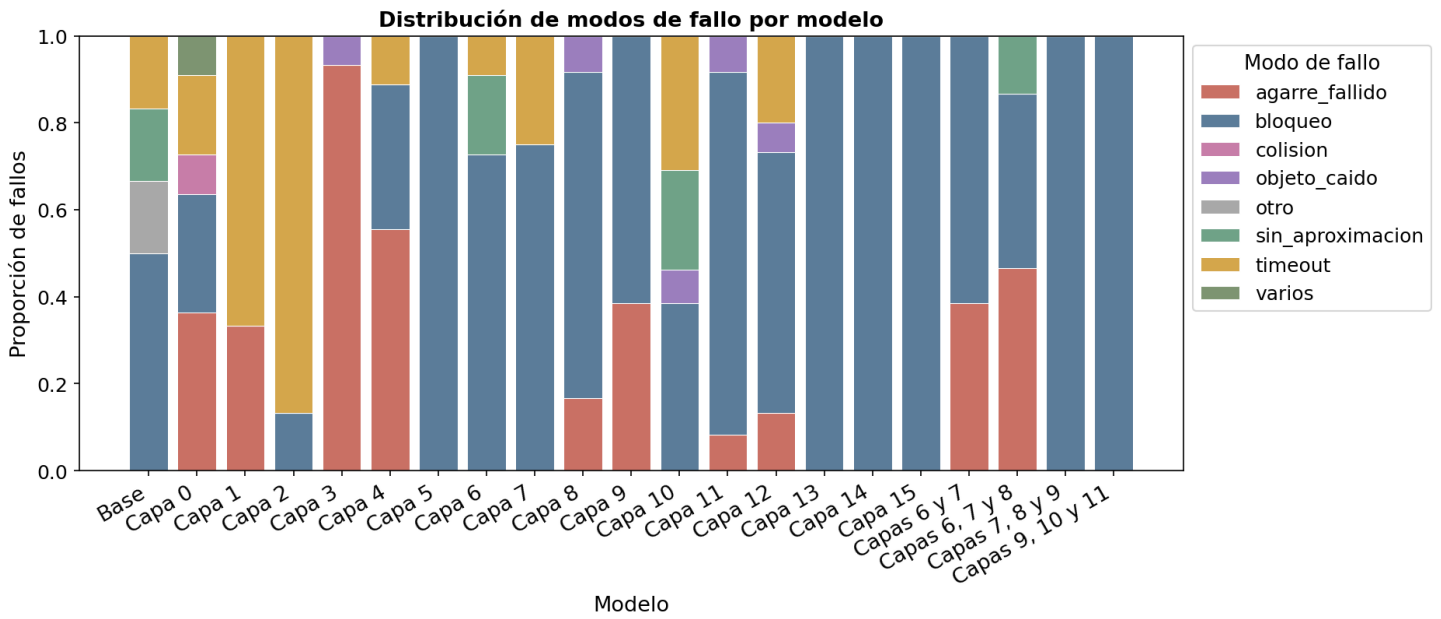


Figura 5.3: Distribution of failure modes observed in the base model, individual ablations, and combinations of contiguous layers.

5.2. Final Evaluation Results

To analyze the practical performance of the proposed architectures on the physical platform, Figure 5.4 synthesizes the mean normalized score of each variant in both experimental conditions (without and with a photograph as a distractor). Meanwhile, Figure 5.5 breaks down the individual behavior in each of the 12 episodes per model, allowing the stability and dispersion of the runs to be observed.

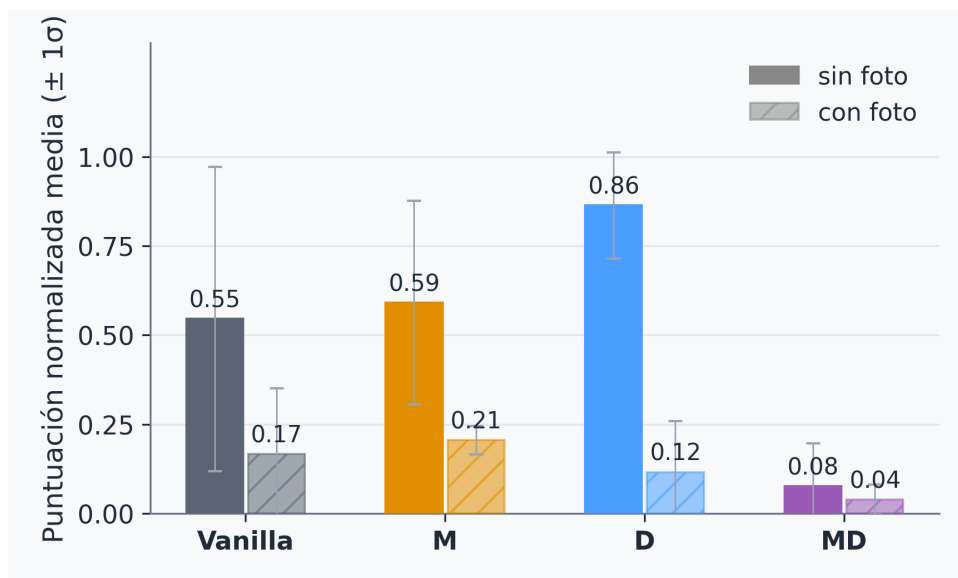


Figura 5.4: Comparison of the mean normalized score by model and experimental condition (without photo and with photo).

When analyzing both graphs, two key conclusions about overall performance are observed.

On the one hand, the photograph distracts all variants equally. Upon including the photo in the scene, the mean score of the different proposals drops drastically (Figure 5.4), and the results become highly unstable from one test to another (Figure 5.5). Except in some isolated case, the presence of the photo prevents the robot from completing the task normally.

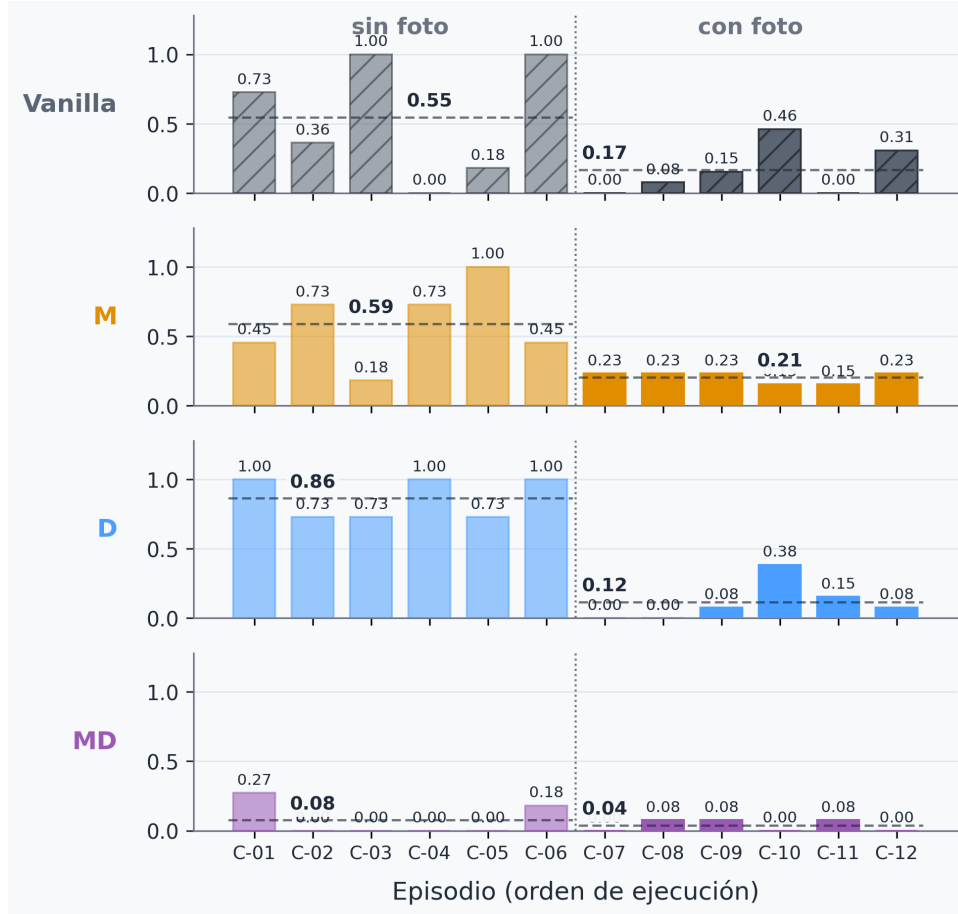


Figura 5.5: Normalized score of each physical run per model. The vertical line separates episodes without a photograph from episodes with a photograph. The horizontal line represents the mean of that set of runs.

On the other hand, SmolVLA-MD does not work and fails completely. In the normal trials without a photo (episodes 1 to 6 in Figure 5.5), SmolVLA-D and SmolVLA-M perform equally to or better than the SmolVLA-Vanilla base architecture, with SmolVLA-D reaching the highest scores in the test. However, when combining both improvements in SmolVLA-MD, the proposal obtains a practically null score in almost all attempts.

This disparity is understood by examining the behavioral indicators in Figure 5.6. While SmolVLA-Vanilla, SmolVLA-M, and SmolVLA-D complete their runs without requiring safety stops, SmolVLA-MD accumulates a high percentage of emergency interventions. During its tests, the policy suffered a complete collapse characterized by tremors, uncontrolled rotations, and markedly erratic and violent movements that

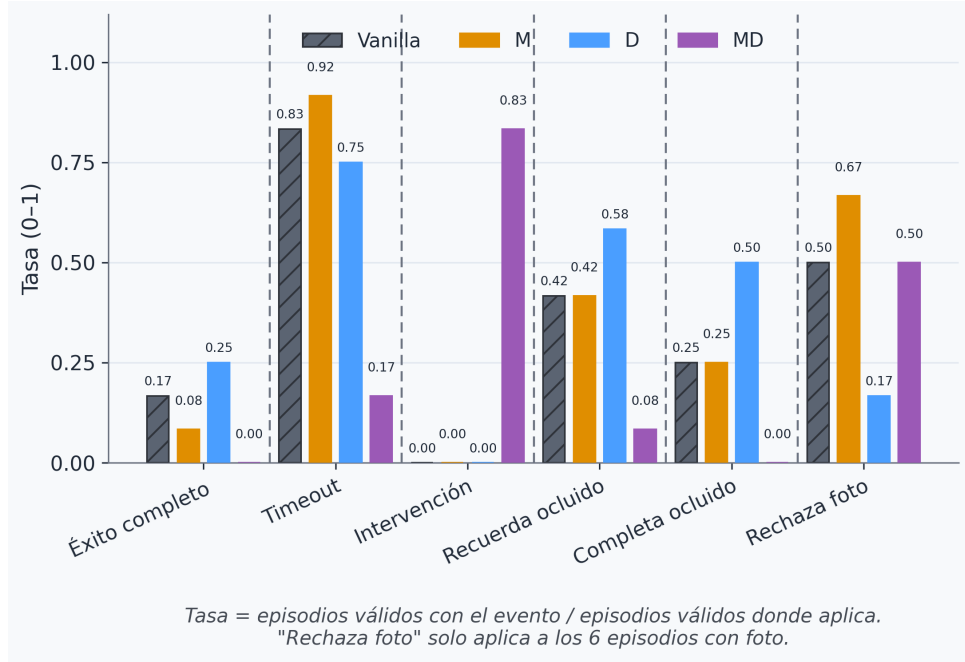


Figura 5.6: Behavioral metrics of the final evaluation: complete success, timeout, emergency intervention, recollection of the occluded object, finishing on the occluded object, and photograph rejection. Derived from execution dynamics and failure modes, these rates allow identifying exactly how each model behaves and why it fails.

forced the arm to be manually stopped to prevent mechanical damage (even registering the breakage of the wrist camera mount in one of the episodes).

In the tests without a photograph, SmolVLA-M performs very similarly to SmolVLA-Vanilla in the memory metrics. Meanwhile, SmolVLA-D is the one that works best in these clean scenes. However, upon adding the photo, SmolVLA-D deteriorates the most out of the three functional variants: Figure 5.6 shows that it almost never manages to ignore the photo, leaving its mean score only above SmolVLA-MD.

5.3. Discussion of the Final Evaluation

The results obtained in the previous section allow analyzing how the variants developed in this work have affected the original SmolVLA model and offering hypotheses about why the observed behaviors occurred.

Starting with SmolVLA-D, its superior performance in distractor-free scenes confirms the hypothesis that injecting depth information into the action expert provides a relevant signal for manipulation. In the absence of an OOD distractor, the geometric representation mitigates the ambiguities of an exclusively RGB image lacking 3D information, improving relative distance estimation and grasping accuracy, in line with the findings described in PointVLA [14]. Likewise, this three-dimensional reference explains its better performance in resolving partial occlusions.

However, the drastic drop of SmolVLA-D when facing the photograph shows that 3D information alone does not guarantee discrimination between real objects and photos.

Although the stereo camera detects a flat surface, the texture match confuses the RGB encoder, causing the network to prioritize the image over the actual geometry. As a result, the network fails to reject the image, indicating that it is necessary to reinforce the priority of the geometric signal over 2D appearance in the architecture.

For its part, the SmolVLA-M variant demonstrated stable behavior, but without a substantial improvement in situations of self-occlusion or loss of visibility compared to the reference variant. The hypothesis that retaining past observations would allow maintaining accuracy when losing sight of the target was limited by the implementation itself. Unlike MEM [12], only a short-term memory with lower temporal resolution was integrated into SmolVLA-M. Without auxiliary losses or more complex attention, the latent representation of the history provided motor stability, but not the spatial guidance necessary to recover the trajectory in moments of occlusion.

The functional collapse of SmolVLA-MD constitutes the most interesting finding of the evaluations. The initial hypothesis assumed that the combination of both architectural improvements would lead to performance superior to the rest of the variants. However, the concatenation and simultaneous injection of both modalities into the action expert resulted in erratic behavior and severe freezings of the control policy.

This phenomenon stems from an interference within the network’s latent space. By introducing the memory vector and the depth adapter at the same time, complexity was increased over a reduced number of layers, hindering the model’s convergence. Although this variant had a shorter training time, the comparison with SmolVLA-M (which operated under similar restrictions without collapsing) shows that the primary cause lies in an incompatibility when integrating both signals into the architecture.

Finally, the models’ inability to correct erratic trajectories when interacting with the distractor (as well as the freezing caused by insisting on grasping the photograph) is directly rooted in the nature of the training set. The majority of the demonstrations consisted of clean executions of the task. Although episodes with corrections after failed grasps on the same target were included, the dataset did not contain any examples where the robot abandoned an object after an unsuccessful attempt to head for a different one.

Consequently, when the policy approached the photograph or suffered a deviation, it lacked action vectors that would guide the model to change targets. This caused the robot to exhaust the episode’s time in endless retries on the image. This behavior demonstrates that robustness in VLA models depends not only on the sensory richness of the architecture but also on the representativeness and variety of the training dataset.

Capítulo 6

Conclusions and Future Work

6.1. Conclusions

This Master’s Thesis has addressed the design, training, and evaluation of 3D perception and short-term visual memory extensions on the lightweight VLA model SmolVLA. Through a procedure that encompassed data collection, an ablation study to select the optimal architecture, and experimental evaluation on a real robot, the following conclusions can be established.

On the one hand, in the ablation test results, the layer set {6, 7, 8} was selected to inject depth perception, chosen for having the least degradation among the candidate options from Phase 2 with more layers removed (see Figure 5.2). We can conclude that the operational objective of this study was successful due to the positive results in the manipulation part of SmolVLA-D, obtaining a score of 0.86 in scenarios without a photograph and a perfect score in occlusion recollection (see Figures 5.5 and 5.6). With this, we can conclude as an extra in this work that, in isolation, it is possible to inject new information into an action expert in a VLA without collapsing the policy.

Likewise, the evidence presented and discussed in Chapter 5 offers uneven support for the hypotheses proposed at the beginning of this work:

For its part, the temporal memory hypothesis H1 has fairly limited support. The temporal window of SmolVLA-M worked in practice as a movement smoother at best, providing stability to the trajectory and achieving a score of 0.59 with 5 out of 6 successes in occlusion memory in the scenario without a photograph, but without actually offering a true improvement when guiding the policy when the target is lost from sight for too long, dropping to a 0.21 score and ending 11 out of the 12 total episodes due to timeouts (see Figures 5.5 and 5.6).

Regarding the spatial perception hypothesis H2, it only holds in clean scenarios. Adding depth in SmolVLA-D served to refine the grasp and resolve simple occlusions, achieving the highest score of the study in the block without a photograph (0.86), but fell short when an out-of-distribution distractor came into play; the network ended up prioritizing the 2D appearance of the RGB over the scene’s actual geometry, dropping to a score of 0.12 in the block with a photograph and managing to avoid grasping the photo in only 1 out of the 6 episodes (see Figures 5.5 and 5.6).

Finally, the synergy hypothesis H3 is refuted. The combination of both modalities in

SmolVLA-MD not only failed to add the best of each variant, obtaining residual scores of 0.08 in scenarios without a photograph and 0.04 in scenarios with a photograph, but it ended up saturating the action expert’s layers and introducing unstable movements that forced 10 manual emergency stops out of 12 attempts. This makes it clear that coupling perceptual branches without a mechanism coordinating how the signals are mixed ruins the control policy’s stability (see Figure 5.4).

In turn, we can conclude that data collection is clearly important, with quality being more important than quantity. Keeping the results in mind, if we look at Figure 4.10 showing the training graph, we can see how the SmolVLA-D and Vanilla variants did reach a plateau where the loss did not decrease, and the other proposals obtained a comparable loss. This indicates that the volume of samples was sufficient to train the models, so the failures respond more to a lack of diversity in the demonstrations. Specifically, sequences with alternative ways of completing the task were missing, such as abandoning one object to move to another, or correcting the execution if a piece is poorly placed during the process.

These results must be interpreted within the work’s own limitations. First, there is a restriction in the experimental scope, having evaluated on a specific set of objects and a specific manipulation task, which limits the ability to generalize the conclusions to other scenarios of greater complexity. Second, the depth perceptual signal was conditioned by the inherent noise of a low-cost stereo camera that could not be calibrated to achieve sub-pixel precision, potentially degrading the quality of the injected geometric representation. Third, the visual memory module was limited by design to a short temporal horizon with low resolution (5 seconds into the past with a 2.5-second stride) to maintain efficiency, proving insufficient to represent prolonged events or long-range dependencies.

On the other hand, there are relevant architectural and computational constraints. The developed extensions were incorporated in a bounded and localized manner within the network, injecting memory before the visual backbone and depth inside the action expert. While this decision allowed preserving the knowledge previously acquired by the base architecture and reducing adaptation costs, it also restricted direct and cross-interaction between the different perceptual improvements. Finally, the high computational cost associated with VLA models limited hyperparameter exploration, training with different initial seeds, and trial repetition, additionally causing the models with memory extensions to be unable to complete their entire training cycle due to strict time and budget constraints.

Ultimately, this work shows that enriching the perception of a lightweight VLA model is not as simple as connecting new signals to the network. For three-dimensional information or temporal memory to translate into greater policy robustness, the architecture must incorporate explicit mechanisms that coordinate how these modalities are weighted within the model, also requiring a sufficiently varied dataset for the network to learn to exploit said improvements.

6.2. Future Work

A first line of future work would be to study in depth how sensory expansion signals are mixed in this type of models. Instead of the direct injection used in this work, it

Conclusions and Future Work

would be advisable to explore different auxiliary losses and fusion methods so that the model learns to rely on this extra information only when necessary, preventing the gradients of the different modules from competing or canceling each other out during training.

When addressing the limitations identified in this study, not all possible courses of action contribute in the same way or require the same investment. The actions with the best cost-benefit ratio are improving data diversity and architectural integration, as they would allow substantially increasing the variants' performance without skyrocketing resources. On an intermediate step would be the review of the memory mechanism and camera quality, which entail a notable cost in VRAM and physical equipment to obtain predominantly perceptual improvements. Finally, the systematic exploration of hyperparameters through successive retrainings represents the most expensive route in time and computation, resulting in the lowest priority option if the main factors are not corrected first.

Finally, it would be highly interesting to evaluate these same extensions on more complex and higher-capacity VLA architectures than SmolVLA, which ultimately remains a model designed for low cost and consumer hardware deployment, as well as extending the experimental tests to tasks of greater complexity and functional variability.

Bibliografía

- [1] M. Shridhar, L. Manuelli y D. Fox, «A Multi-Task Transformer for Robotic Manipulation,» en, sep. de 2022. dirección: <https://arxiv.org/abs/2209.05451>.
- [2] T. Z. Zhao, V. Kumar, S. Levine y C. Finn, *Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware*, en, arXiv:2304.13705 [cs], abr. de 2023. DOI: 10.48550/arXiv.2304.13705. dirección: <http://arxiv.org/abs/2304.13705>.
- [3] A. Brohan et al., *RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control*, en, arXiv:2307.15818 [cs], jul. de 2023. DOI: 10.48550/arXiv.2307.15818. dirección: <http://arxiv.org/abs/2307.15818>.
- [4] O. M. Team et al., *Octo: An Open-Source Generalist Robot Policy*, en, arXiv:2405.12213 [cs], mayo de 2024. DOI: 10.48550/arXiv.2405.12213. dirección: <http://arxiv.org/abs/2405.12213>.
- [5] K. Black et al., π_0 : A Vision-Language-Action Flow Model for General Robot Control, en, arXiv:2410.24164 [cs], ene. de 2026. DOI: 10.48550/arXiv.2410.24164. dirección: <http://arxiv.org/abs/2410.24164>.
- [6] M. J. Kim et al., *OpenVLA: An Open-Source Vision-Language-Action Model*, en, arXiv:2406.09246 [cs], sep. de 2024. DOI: 10.48550/arXiv.2406.09246. dirección: <http://arxiv.org/abs/2406.09246>.
- [7] NVIDIA et al., *GR00T N1: An Open Foundation Model for Generalist Humanoid Robots*, en, arXiv:2503.14734 [cs], mar. de 2025. DOI: 10.48550/arXiv.2503.14734. dirección: <http://arxiv.org/abs/2503.14734>.
- [8] P. Intelligence et al., $\pi_{0.5}$: a Vision-Language-Action Model with Open-World Generalization, en, arXiv:2504.16054 [cs], abr. de 2025. DOI: 10.48550/arXiv.2504.16054. dirección: <http://arxiv.org/abs/2504.16054>.
- [9] J. Wen et al., *TinyVLA: Towards Fast, Data-Efficient Vision-Language-Action Models for Robotic Manipulation*, en, arXiv:2409.12514 [cs.RO], sep. de 2024. DOI: 10.48550/arXiv.2409.12514. dirección: <https://arxiv.org/abs/2409.12514>.
- [10] M. Shukor et al., *SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics*, en, arXiv:2506.01844 [cs], jun. de 2025. DOI: 10.48550/arXiv.2506.01844. dirección: <http://arxiv.org/abs/2506.01844>.
- [11] C. Zhang, P. Hao, X. Cao, X. Hao, S. Cui y S. Wang, *VTLA: Vision-Tactile-Language-Action Model with Preference Learning for Insertion Manipulation*, en, arXiv:2505.09577 [cs], mayo de 2025. DOI: 10.48550/arXiv.2505.09577. dirección: <http://arxiv.org/abs/2505.09577>.
- [12] M. Torne et al., «MEM: Multi-Scale Embodied Memory for Vision Language Action Models,» en, mar. de 2026. dirección: <https://www.pi.website/download/Mem.pdf>.

- [13] Y. Ze, G. Zhang, K. Zhang, C. Hu, M. Wang y H. Xu, *3D Diffusion Policy: Generalizable Visuomotor Policy Learning via Simple 3D Representations*, en, arXiv:2403.03954 [cs], sep. de 2024. DOI: 10.48550/arXiv.2403.03954. dirección: <http://arxiv.org/abs/2403.03954>.
- [14] C. Li, J. Wen, Y. Peng, Y. Peng e Y. Zhu, «PointVLA: Injecting the 3D World Into Vision-Language-Action Models,» en, *IEEE Robotics and Automation Letters*, vol. 11, n.º 3, págs. 2506-2513, mar. de 2026. DOI: 10.1109/LRA.2026.3653303. dirección: <https://ieeexplore.ieee.org/document/11346992/>.
- [15] P. Holderrieth y E. Erives, *An Introduction to Flow Matching and Diffusion Models*, MIT Class 6.S184: Generative AI With Stochastic Differential Equations, jul. de 2025. arXiv: 2506.02070 [cs.LG]. dirección: <https://arxiv.org/abs/2506.02070>.